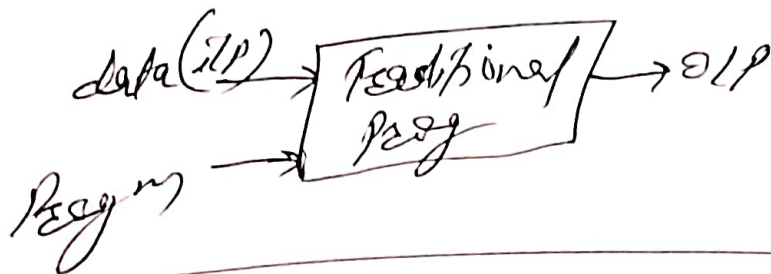


mlc learning

ML $\Rightarrow$ It is the field of study that gives computers the capability to learn without explicitly prog.



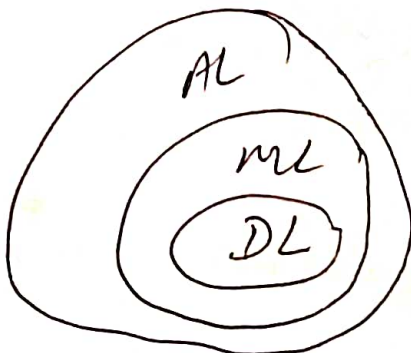
let the mlc learn & adapt for the user

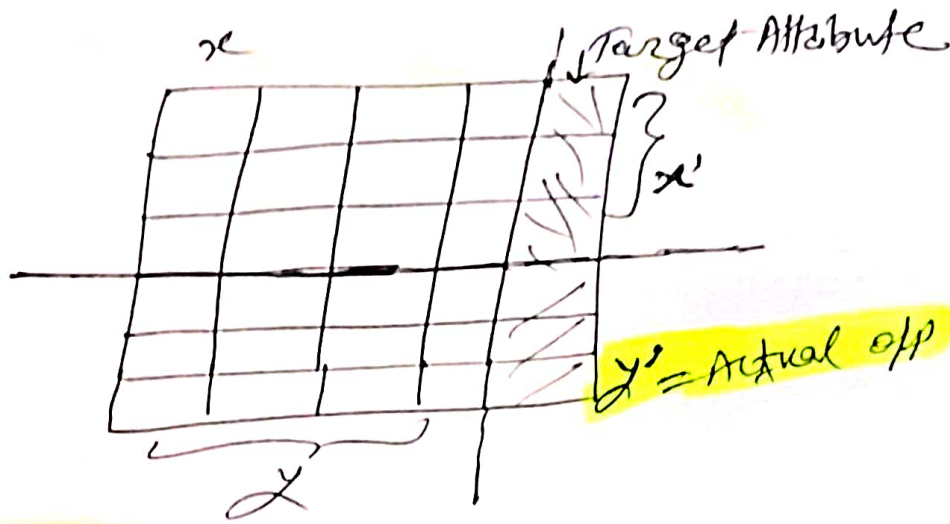
Ex $\Rightarrow$ online shopping

objective of ~~error~~ mlc learning
is lesser the Error margin

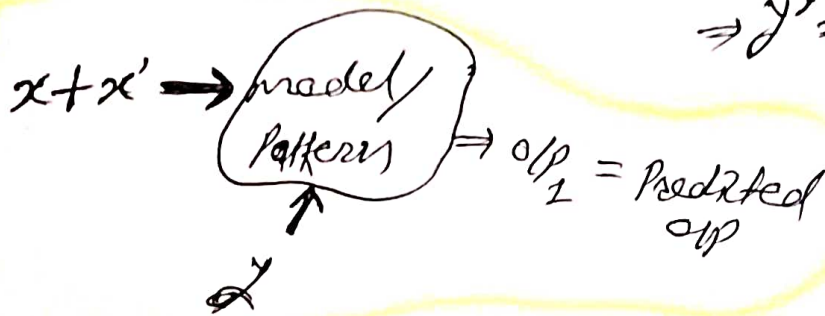
$$\text{Error margin} = \text{Actual O/P} - \text{Predicted O/P}$$

when $\text{Actual O/P} = \text{Predicted O/P}$ = same then objective of mlc is Achieve





$\Rightarrow y' = \text{used for check}$



$\rightarrow y'$ never be used in model process, only use for checking

$\rightarrow \text{check} = \text{Comparison betn Actual o/p \& Predicted o/p}$

Accurate model = Actual & Predicted o/p are same
 100% (or)

Error margin = 0

(or) $\text{Actual o/p} - \text{Predicted o/p} = 0$

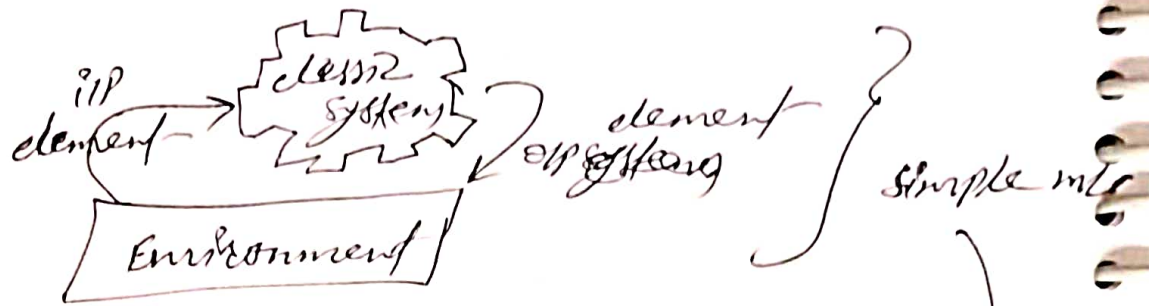
$\Rightarrow$ if model = 100% Accurate then Zero Error

$\Rightarrow$ if model $\neq$ 100% Accurate then Errors are Present.

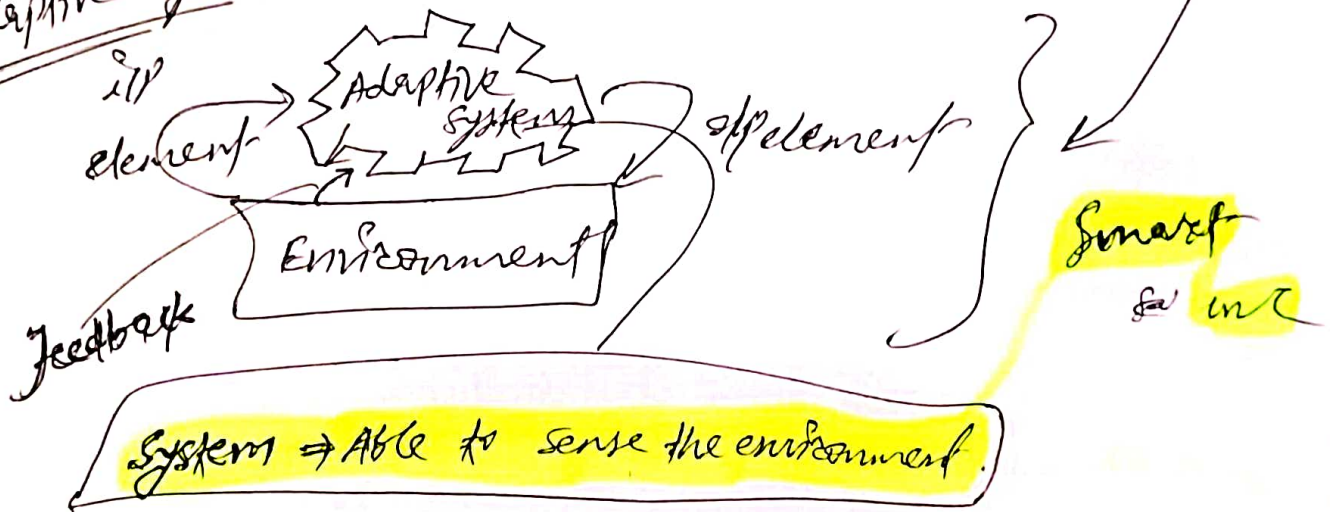
Comparing ($o/p_2 == y'$)

Non-Adaptive * Class2 & Adaptive m/c

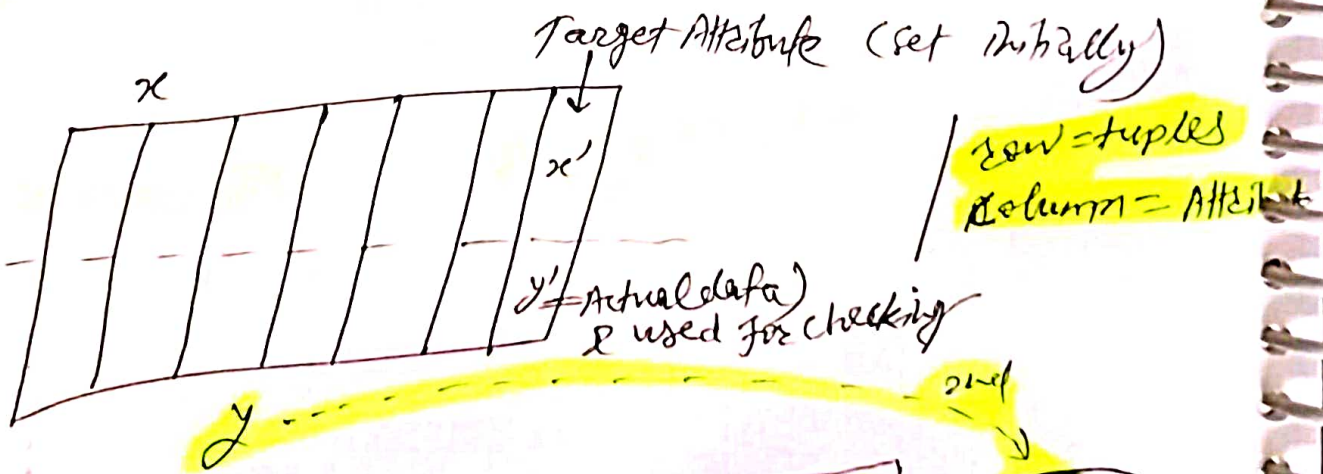
(I) Class2 / non-Adaptive system = static = primitive



(II) Adaptive system



* Training & Testing (first step is training & 2nd step is Test)



Training phase $\Rightarrow x + x' \xrightarrow{\text{1st}} \text{model} \xrightarrow{\text{2nd}} \text{op}$
 model = is build from training data
 pattern / log2
 Predicted data



low variance, high bias
underfitting = very less features & use for model training



overfitting = Very more features use for model training
high variance, low bias

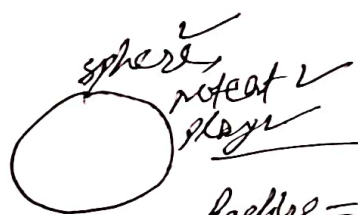


Ex $\Rightarrow$ Problem $\Rightarrow$ Identification of Ball

Parameters

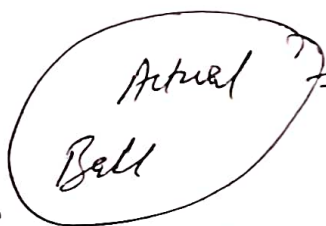
- ① sphere
- ② Not BAT
- ③ Play
- ④ Radius = 0.5cm

} Training with these parameters



Test Ball

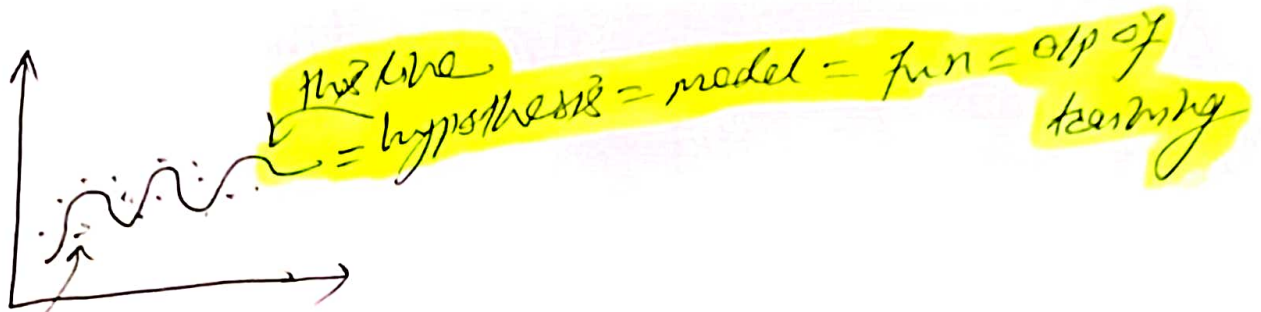
Radius = 0.5 X



Actual Ball $\neq$ model Prediction
wrong

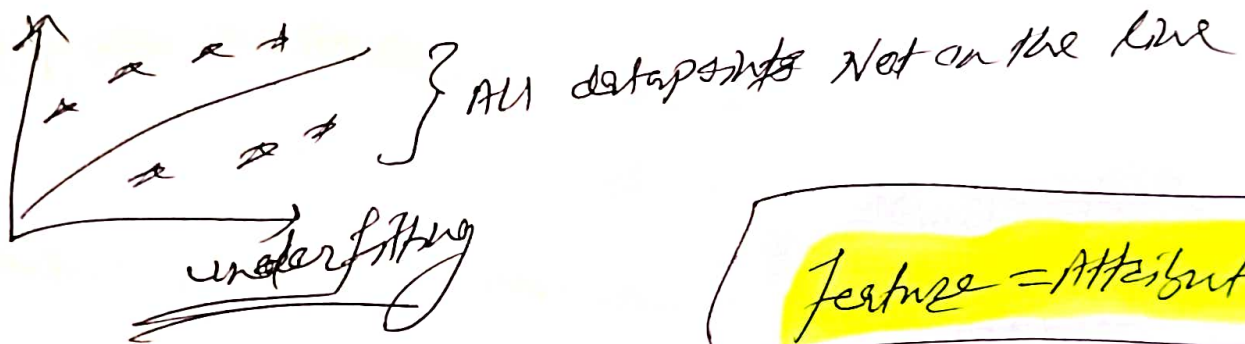
$\Rightarrow$ Even though original Ball
But due to radius less
or additional parameter
model predicted wrong.

- $\rightarrow$ cricket Ball
- $\rightarrow$ Basket Ball
- $\rightarrow$ Ludo Ball



data points = observation point

overfitting = when model try to predict all data point on the line (very close to the line)
 → over crowd of datapoint on the line



feature = Attribute

Ex ⇒ if shape is sphere then it is a Ball
 Train →

○ → Ball
 sphere
 ↳ only one feature

Test →

orange → Ball } × Prediction
 sphere

∴ more feature require for correct result
underfitting

Knn $\rightarrow$ Euclidean distance

2 cases $\rightarrow$ Test Train } difference between Test Train
if smaller then good
difference should be less

- 1) Random state
- 2) Samples
- 3) Parameters
- 4) test & train size

Accuracy depends

F_T	T_T
F_F	T_F

$$\frac{TP}{TP+FP}$$

Recall $\frac{TP}{TP+FN}$

$$\frac{TP}{TP+FN}$$

$$\frac{N}{TP+FN}$$

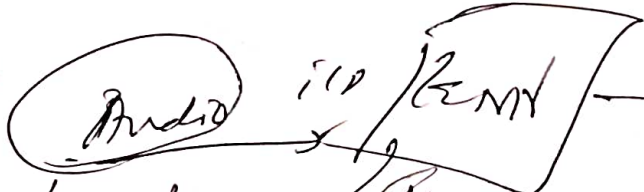
$$F1 = \frac{P+R}{2}$$

Independ

actual

- 1) librosa = plot
- 2) sklearn = Training
- 3) numpy = prediction

depend



CPU time = 5 min 9 sec

Baseline of old model labeled

100% result

$P = \frac{\text{Predicted class} \cap \text{Observed}}{\text{Predicted class}}$

* Feature selection Technique

Feature = Attribute

① Filter method = (relevance of Attribute)

Raw data = Raw data

- χ^2 (Infer gain method)
- χ^2 - square Test
- Correlation coefficient

optimal
Features = Relevant
Features

② Wrapper methods = Expensive & overfitting

- Recursive feature elimination
- Genetic Algorithms

$$\chi^2 \propto \sum \frac{(O_i - E_i)^2}{E_i}$$

③ Embedded methods

→ Decision Tree

→ if any Attribute highly affect or related to Target Attribute then that Attribute is more attribute & must be selected

④ Target Attribute is depend on any of Attribute then must be selected

Find the usefulness of ~~each~~ attribute
no or less overfitting

model = o/p generated from Training

$$\chi^2 \propto \sum \frac{(A - P)^2}{P}$$

Training Dataset

→ Algo

→ model

observed
= data
actual data
(n)

* one vs one multi classification

$\text{num (class)} \approx \text{num (classification)}$

(3)

R = Coefficient of correlation (varies betn -1 & +1)

R^2 = Coefficient of determination

$R=0$ means no correlation

$$R^2 = R * R$$

⇒ Coefficient of determinant = ^{multiple of} Coefficient of correlation

⇒ Good & Bad R -values depends on context.

Good R^2 values = 0.50 to 0.99

- ⇒ Multiple R-squared
- ⇒ Adjusted R-squared
- ⇒ F-statistic
- ⇒ P-value

$$R^2 = 1 - \frac{\sum (y_i - \hat{y})^2}{\sum (y_i - \bar{y})^2}$$

where $\hat{y}$ = Predicted value
 $\bar{y}$ = mean value of y

Predicted
mean

Residuals = difference betn Actual series & Predicted values

R -value measures = strength of correlation

p -value measures = how likely you would be to observe a correlation of this strength under the null hypothesis.

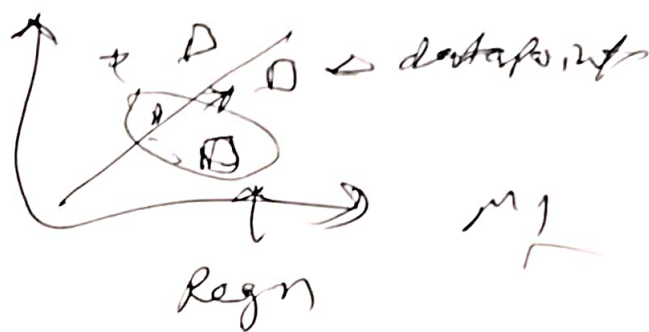
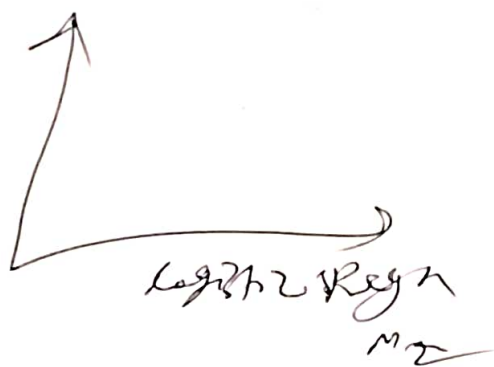
* Confusion matrix

↓
use to measure the performance of the classification model.

		Actual value	
		(+)	(-)
Predicted value	(+)	TP	FP
	(-)	FN	TN

→ It is performance evaluation tool
 → represent the Accuracy of model.

2x2
 It is Confusion matrix



m_3 m_4 m_5
 (2) $\rightarrow$ Best model identifier of R
Confusion matrix

{ Precision
Recall

1) Cost Function

Cost Function

2) Step Function

Back propagation
 $\hookrightarrow$ optimizer $\rightarrow$ RMS Prop

Ada grad, Adam

Error
MSE

1) Under fit
 Variance
 bias

2) over

Note $\Rightarrow$ TP & TN means the Predicted value matches the Actual value.

② ^(FP) Type-I error happens when model makes Incorrect Prediction $\hat{=}$ it predicted possible for Actual (-) value.

③ ^(FN) Type-II error happens when model makes incorrect Prediction of an Actual (+)ve values

Loss funⁿ

As (-)ve

MSE $\Rightarrow$ Mean Absolute Error = (L1 Loss)

$\hookrightarrow$ It measures the Avg magnitude errors.

MSE $\Rightarrow$ Mean Squared Error = (L2 Loss = Quadratic Loss)

$\hookrightarrow$ It measures the Avg squared magnitude of errors.

RMSE $\Rightarrow$ Root mean squared Error =

$\hookrightarrow$ It measures squared root of the Avg squared magnitude

Mean Bias Error $\Rightarrow$ possibility of (-)ve value of errors.

$$MAE = \frac{1}{n} \sum_{t=1}^n |e_t|$$

$$MSE = \frac{1}{n} \sum_{t=1}^n (e_t)^2$$

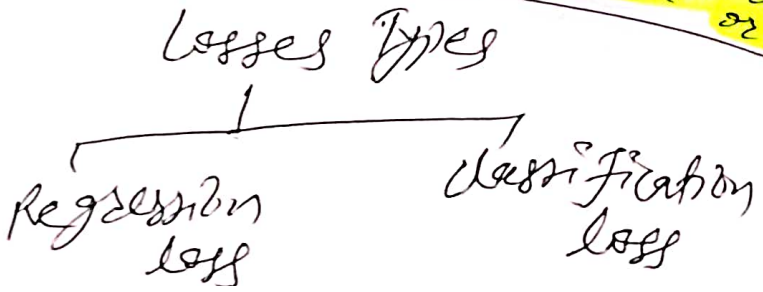
$$RMSE = \sqrt{\frac{1}{n} \sum_{t=1}^n (e_t)^2}$$

$$MBE = \frac{1}{n} \sum_{t=1}^n e_t$$

only (+)
Positive

$n = \text{num (observations)}$
or num (samples)

Both (+) & (-)



$$e_t = \text{Actual value} - \text{Predicted value}$$
$$e_t = (y_i - \hat{y}_i)$$

(4)

	Actual value	
	(+)	(-)
Predicted value	TP	FP
	FN	TN

→ CM compare Actual value with Predicted value.

→ It show how well our classification model is performing

fig → Binary classification model → " — what kind of errors it is making

Column = Actual value

Row = Predicted value

TP = True Positive = Predicted value matches Actual value
 (↑) (↑)
 (+) (+)
 True True

TN = True Negative = — " —
 Predicted value (-) & Actual value (-)

FP = False Positive = Type I error

→ Predicted value was falsely Predicted
 → Actual value = (-), but model Predicted (+)

FN = False (-) = Type II error

→ Predicted value was falsely Predicted
 → Actual value was (+) but Predicted (-)

class data points ⇒ numerical values of TP, TN, FP, FN

Note ⇒ Based on the confusion matrix other metrics are evaluated like Accuracy, Precision, Recall, F1 score

Un-informed Search

Search without info

No knowledge

Time consuming
more complexity
(Time, space)

DFS, BFS

Blind (state)
Search

→ Blindly move

→ Brute Force
method

Travelling Salesman
Problem

→ give minimal soln
But take more time

Where Problem
as states growth
Exponential.

give ^{Not} Guarantee of
optimal soln.

* Informed Search

① Search with info

② use knowledge to find
steps to solutions

③ Quick solution

④ less complexity (Time, space)

⑤ AA, Heuristic DFS, BFS

⑥ moving to known
direction = Heuristic

⑦ Heuristic = discovery =
Problem solving by
experimental

use when we want soln of state growth
in exponential power
Solving it, NP Problem → need more
solve in polynomial time / time, cost, space

⑧ ~~not so~~ optimal may be
⑩ Guarantee of ~~not so~~

⑪ Heuristic funn = $h(n)$

RMSE $\equiv$ It gives Absolute term which tell how well model can predict

$R^2 \equiv$ It gives Variation in response variable

Confusion Matrix

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + TN + FP} = \frac{\text{True}}{\text{True} + \text{False}}$$

only (+)
 $\uparrow$ precision = $\frac{TP}{TP + FP}$

$$\text{Recall} = \frac{TP}{TP + FN}$$

Actual value

	+	-
Predicted values +	TP	FP
-	FN	TN

Fig 1.3 Confusion matrix

$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Stochastic Gradient Descent (SGD)

- ① uses a single random sample at each iteration.
- ② Process fewer data, $\therefore$ less expensive.
- ③ Fast convergence due to frequent updates.
- ④ High noise due to frequent updates.
- ⑤ less stable.
- ⑥ Req^d less memory.
- ⑦ less sensitive to initial parameter.

(BGD) Batch Gradient Descent

- ① uses the entire dataset at each iteration.
- ② more expensive, as it processes entire dataset.
- ③ slow convergence, due to less frequent updates.
- ④ low noise as it updates parameters using all data points.
- ⑤ more stable.
- ⑥ Req^d more memory.
- ⑦ more sensitive to initial parameter.

Epoch $\Rightarrow$ one complete pass of the training dataset through the algorithm.

$\rightarrow$ Epochs number $\equiv$ hyperparameter

Iteration $\Rightarrow$ num of batches req^d to complete one epoch.

Batch size $\Rightarrow$ num of training examples utilized in one iteration.

Bias $\equiv$ Training Data

Variance $\equiv$ Test Data

High bias = more error rate (of Training data) with respect to regression line.

underfitting = Training Data is high.

Low bias = low error rate of training data.

A* Algo $\rightarrow$ Heuristic
 $\rightarrow$ Informed Search

Admissible
Algo

guarantee of
optimal sol'n

$$f(n) = g(n) + h(n)$$

Actual cost
from start node
to 'n'

Estimate cost from
n to Goal node.

AI imp points

(1)

(1) Inventor of AI = John McCarthy

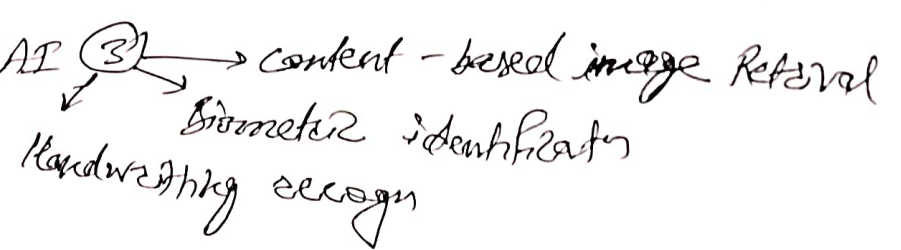
(2) AI aiming to develop intelligent m/c.

(3) Blind search = small search space = uninformed search

(4) when m/c capable of changing its course of action based on the external environment without any external help then m/c called "Intelligent"

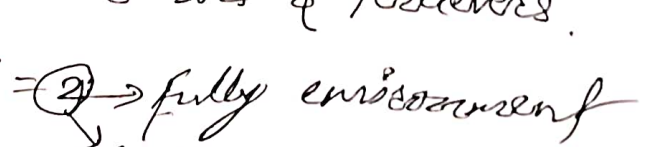
(5) AI Category (3) 

(6) Face recognition system based on Applied AI ^{Approach} ~~Search~~, used for smart system

(7) Recognition system in AI (3) 

(8) Turing Test = The imitation game.

(9) AI Agent can interact with its environment by using sensors & perceivers.

(10) observing environments in AI = (2) 

(11) Deterministic environment considered as strategic.

(12) Funⁿ of AI Agent = map the percept sequence to an Action.

(13) In log file = Errors & Bugs are recorded.

(14) Time sharing OS = Allow users to use one system with two different terminals.

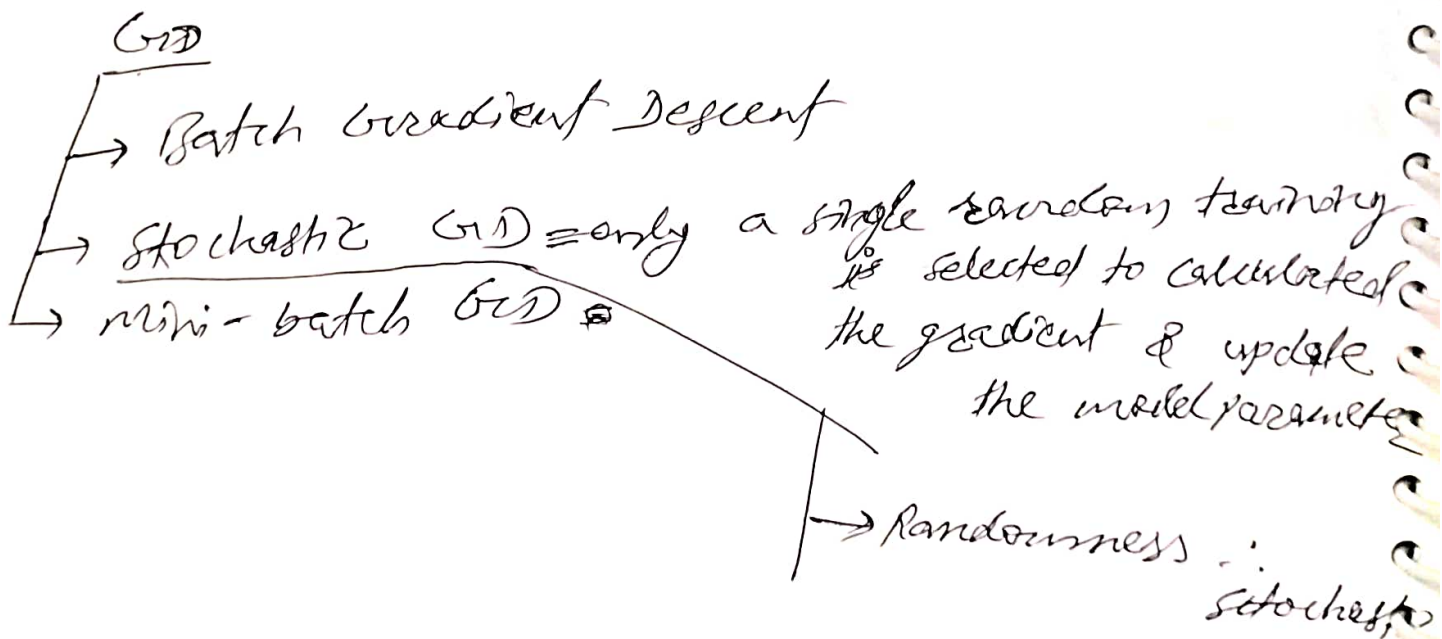
(15) Batch OS = similar type of tasks are group together.

Adv of GD

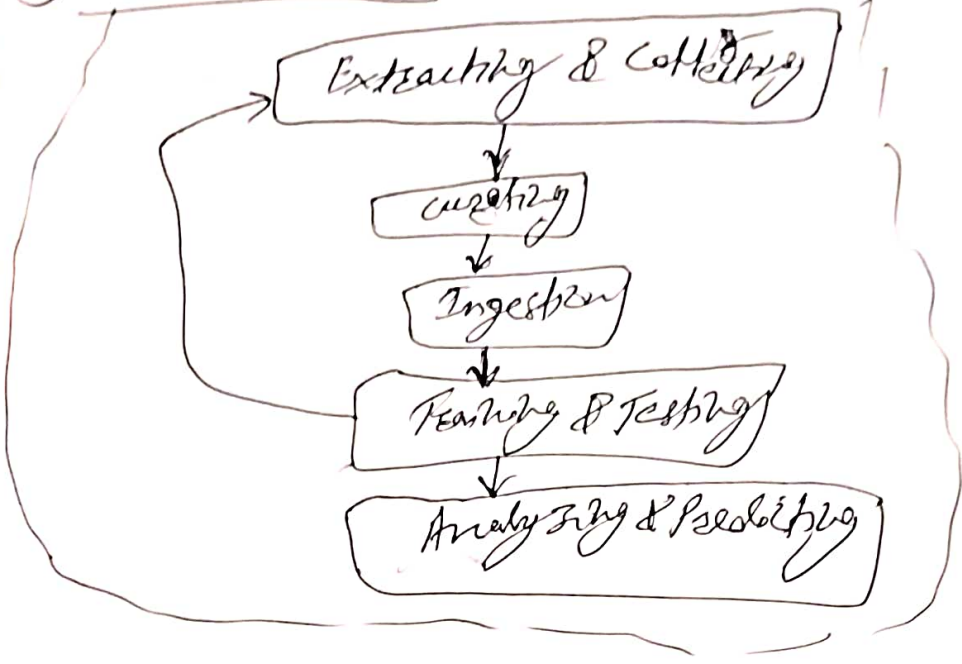
- ① Flexibility
- ② Scalability
- ③ Convergence = to move or tend towards one point or one another.

Dis of GD

- ① Sensitive to learning rate = low learning rate can make the algorithm converge slowly
- ② Slow Convergence $\rightarrow$ GD req^s more iteration to converge to the minimum
- ③ Local minima $\rightarrow$ GD stuck if funⁿ has multiple local minima
- ④ Noisy updates $\rightarrow$ updates in GD are noisy & have a high variance



* Framework of AI



* DIB & Knowledge Base

(2)

- ① Collⁿ of data in db represents the facts.
- ② operate on single object.
- ③ All info needed to be explicitly stated.
- ④ DIB are maintained for operational purpose.
- ⑤ Repⁿ by relational, NW or hierarchical model.
- ⑥ Interactⁿ with database by transactⁿ prog & report generator.
- ⑦ It consist of info at higher level of abstractⁿ.
- ⑧ operates on a class of objects.
- ⑨ knowledge base has the power of inference.
- ⑩ ⁴⁷⁸ used for planning & data analysis.
- ⑪ KR is by logic or rules or frames or semantics nets.
- ⑫ KR has consultation with system & provides needed data to obtain the solⁿ.

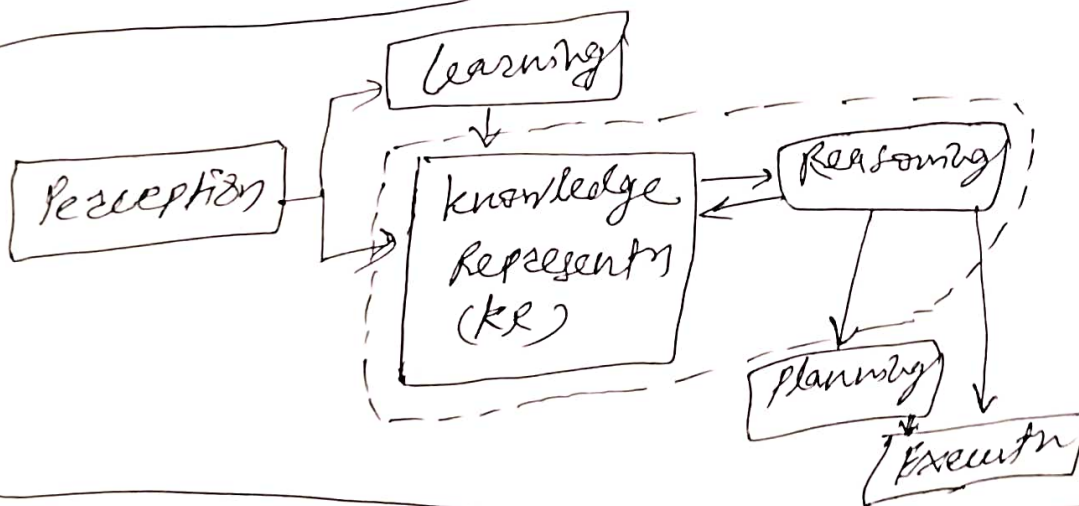
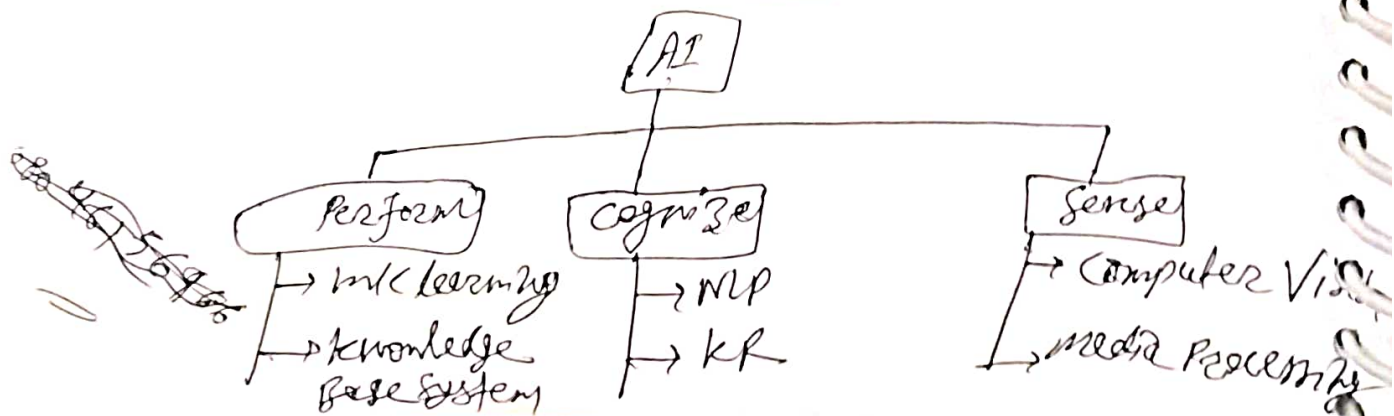


fig → AI cycle



machine learning types $\left\{ \begin{array}{l} \rightarrow \text{supervised} \\ \rightarrow \text{unsupervised} \\ \text{semi-supervised} \rightarrow \text{Reinforcement} \end{array} \right.$

(1)

Tasks of ml

- ① Regression
- ② Classification
- ③ Clustering
- ④ Decision-making
- ⑤ Neural net
- ⑥ Dimensional Reduction

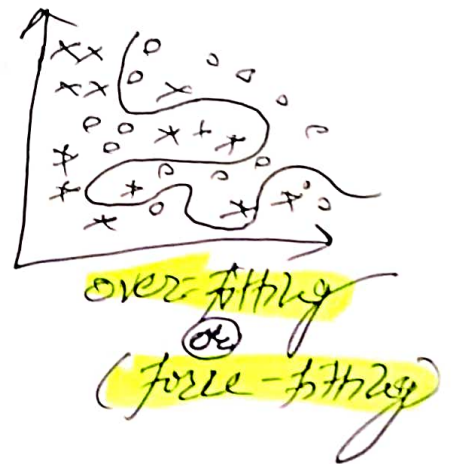
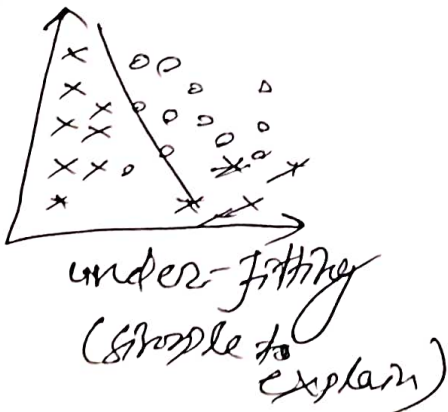
ml model = Is a program that can find a patterns or make decisions from a previously unseen dataset.

Training will be done on known dataset

overfitting $\rightarrow$ High variance & low bias $\approx$ low training error $\rightarrow$ high error rate in ^{Test} data.

$\rightarrow$ It is undesirable ml behavior.

$\rightarrow$ It occurs when ml model gives accurate result for existing (known data) but not to the new data.



$\Rightarrow$ Regularization is ml use to prevent overfitting by adding penalty terms to the loss function.

* Semi-supervised learning

→ Build a model through a mix of labeled & unlabeled data.

Algorithms

① GAN = Generative Adversarial net

Appⁿ Ex → Audio & video manipulation
→ Data creation.

② Self-trained Naive Bayes

Appⁿ Ex → NLP

* Reinforcement learning

→ self-interpreting but based on a system of rewards & punishments learned through trial & error.

→ It focus on maximum rewards.

Appⁿ Algorithms

① D-learning

Appⁿ Ex → policy creation

→ consumption selection

② model-based value estimation

Appⁿ Ex → linear tasks

→ Estimating parameters

* Supervised Learning

→ Data Scientist Provides in, out & feedback to build model.

Algorithms

① Linear Regression

Appⁿ Ex → Sales Forecasting
→ Risk Assessment

② SVM

Appⁿ Ex → Image Classification
→ Financial Performance Comparison

③ Decision Tree

Appⁿ Ex → Predictive Analytics
→ Pricing

* Unsupervised Learning

→ use deep learning to arrive at conclusions & patterns through unlabeled training data.

Algorithms

① Association → Ex Appⁿ Sales Funnel, Word Association & Searcher.

② k-means clustering

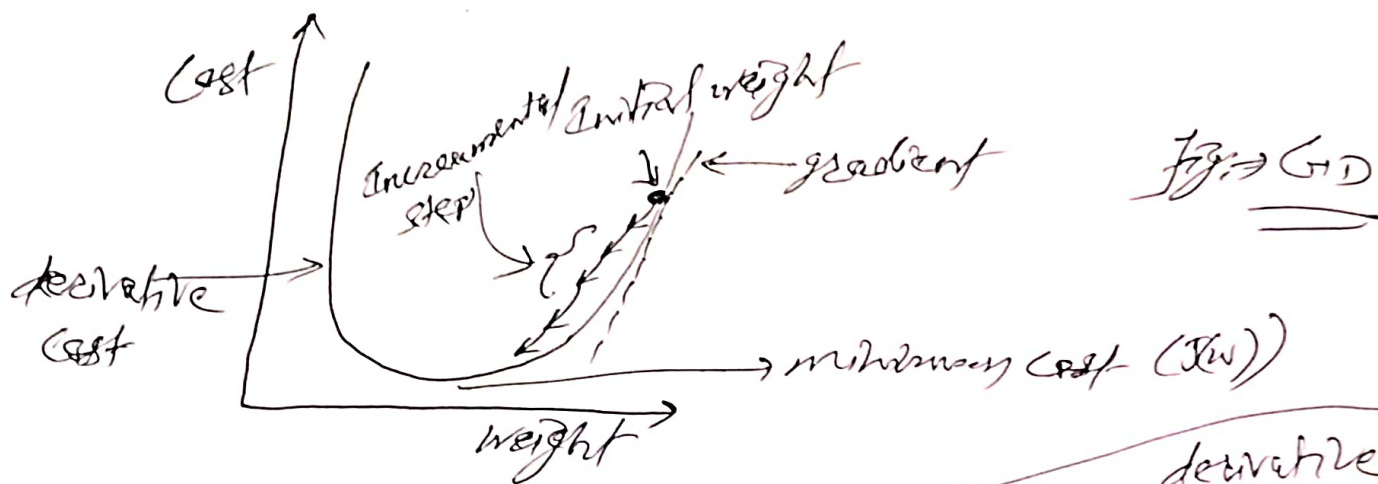
Ex Appⁿ → performance monitoring, searcher tab inter.

③ ANN

Ex Appⁿ → Generate New, Synthetic data, Pattern Recognition, Data Mining.

RNN $\rightarrow$ $dw =$ Gradient Vanishing &
Exploding gradient.

- * Gradient Descent (GD) → Find parameters with the highest accuracy on Training & Testing.
- Iterative first order optimisation algo.
- It used to find a local minimum/maximum of a given fun.



$$\text{New weight (w)} = \text{Old weight (w)} - \underset{\substack{\uparrow \\ \text{learning rate}}}{\alpha} \frac{\partial}{\partial w} \underbrace{J(w)}_{\substack{\text{derivative of} \\ \text{error of} \\ \text{weight} \\ \text{minimum}}}$$

minimum cost $\equiv$ Point of convergence = where the cost fun is at its minimum

Note → In ML, Accuracy is very important.

→ Based on the difference betn model prediction & actual data points we find parameter of model for better Accuracy

→ To find parameters we apply Gradient descent on Cost fun on ML model

* Dimensionality Reduction Technique

Dimensionality Reduction Technique

Feature selection

Dimensionality Reduction

Feature Extraction Technique

Principal Component

Analysis $\Rightarrow$

Statistical process that converts the observations of correlated features into set of linearly uncorrelated features with the help of orthogonal transformations.

New transformed features are called Principal Component.

Applications of Principal Component Analysis

- ① movie Recommendation systems
- ② optimizing the power allocation in various communication channels.

Component's
② factor based

Projection Based

- Factor Analysis
- PCA = Principal Component Analysis
- Independent Component Analysis

- ISOMAP
- UMAP

PCA is based on

- ① variance & co-variance
- ② Eigen values & Eigen Factors

<u>Parameters</u>	ANN	CNN	RNN
Types of Data	Tabular or Text data	Image Data	Sequence data
Parameter sharing	No	yes	yes
Fixed length I/O	yes	yes	No
Recurrent connection	No	No	yes
Vanishing & Exploding Gradient	yes	yes	yes
Spatial relationship	No	yes	No
Performance	less powerful	more powerful	In betw ANN & CNN
Appln	Face Recogn, CV	NLP, Text digitization	Text to speech
Advantage	Fault tolerance, Ability to work with incomplete knowledge	High Accuracy, weight sharing	Conversion
D/W	HW dependence, unexplored new behavior	large training data needed	Remembers each & every info Time series Prediction

CNN is most powerful

AI → Artificial = manmade
→ Intelligence = power of thinking

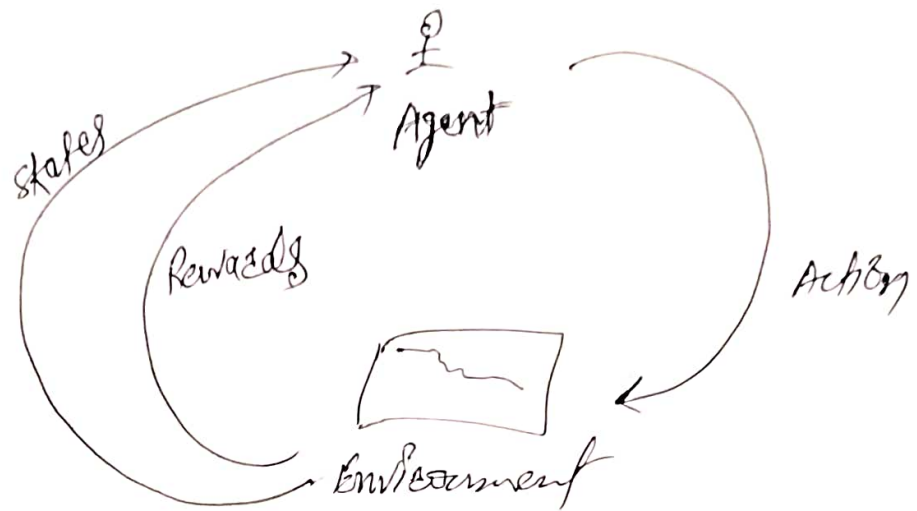
Reasons of AI

- SW or device can be made to solve Realtime Problems
- creation of Virtual Assistant (Alexa, Siri)
- Robots development
(helps in dangerous environment)

Goals of AI

- 1) Replication of human Intelligence
- 2) Solving Problems - knowledge based
(Previous knowledge
∴ Training Dataset)
- 3) Playing theorems, chess, self driving car.
- 4) Providing Advice → Virtual Assistant
- 5) Intelligence comes betⁿ Perception & Action.

A Reinforcement Learning



Tensor Flow

Free, open source soft library for ML & AI.

used to develop models for various task including NLP, image recognition.

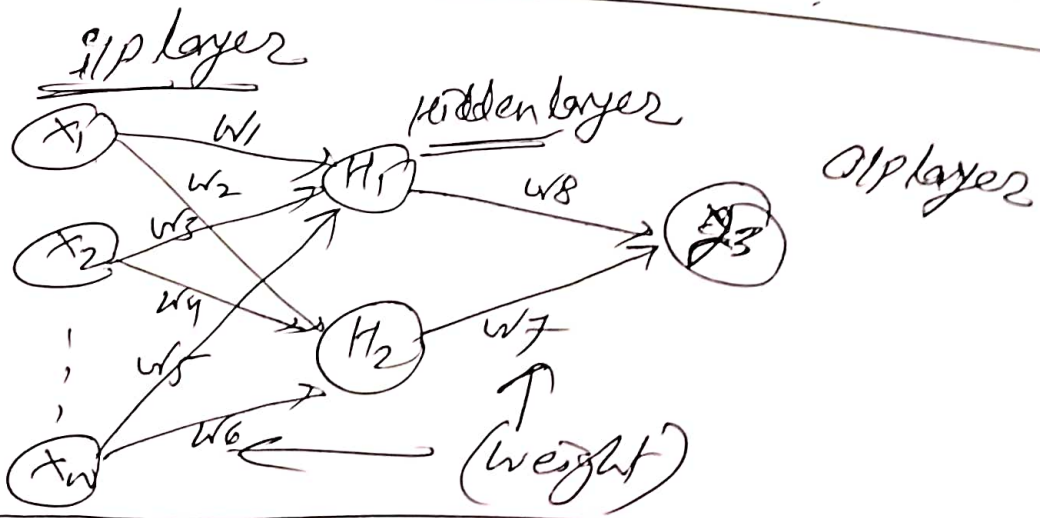
used to train ml learning models

Tensor Flow $\Rightarrow$ Tensor = multidimensional data array

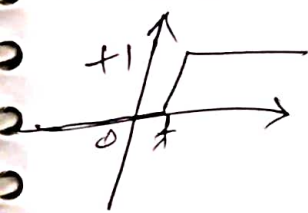
operations performed on the neural net performs on Tensors

It is used to build & train deep learning models as it facilitates the creation of graph & efficient execution on various hw platform.

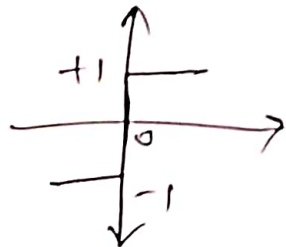
ANN $\Rightarrow$



⑥ Step Functn = unit functn (0 to 1)



⑦ Sign Functn = (-1 to +1)



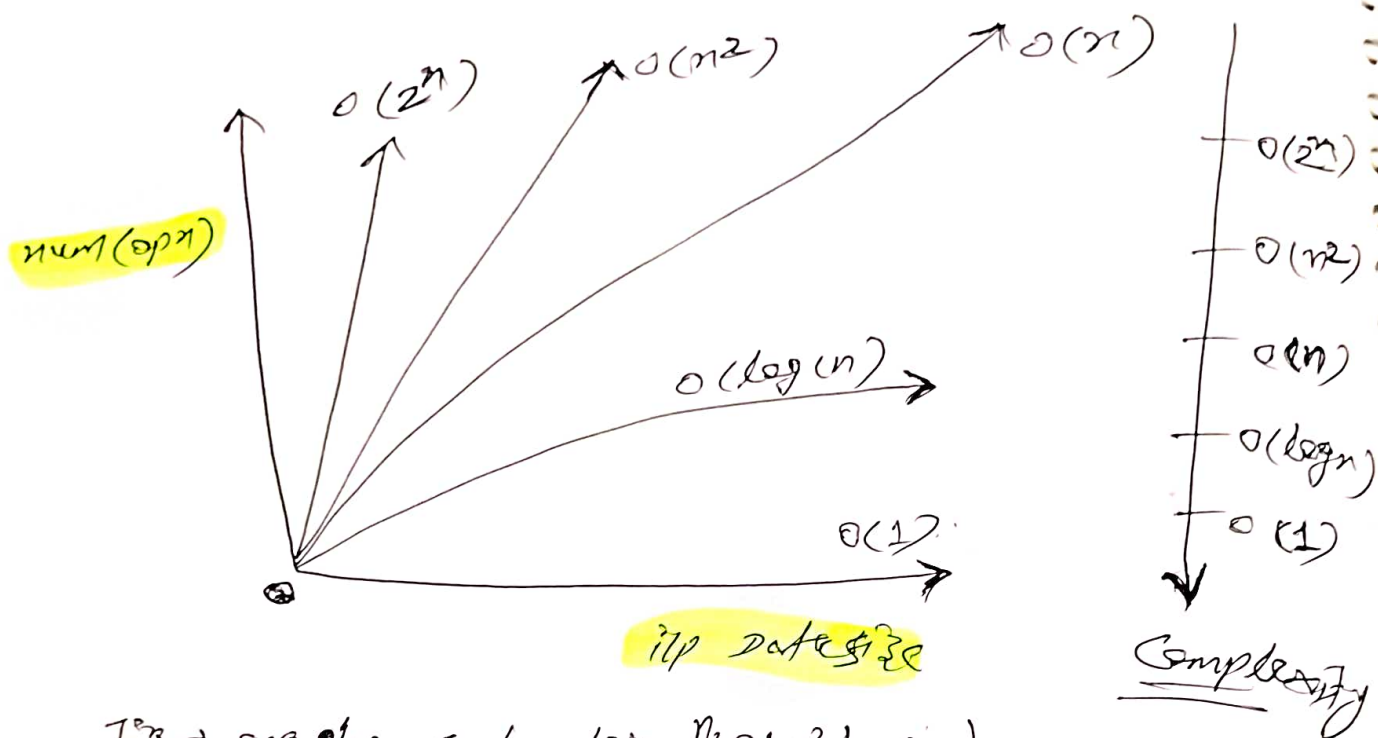
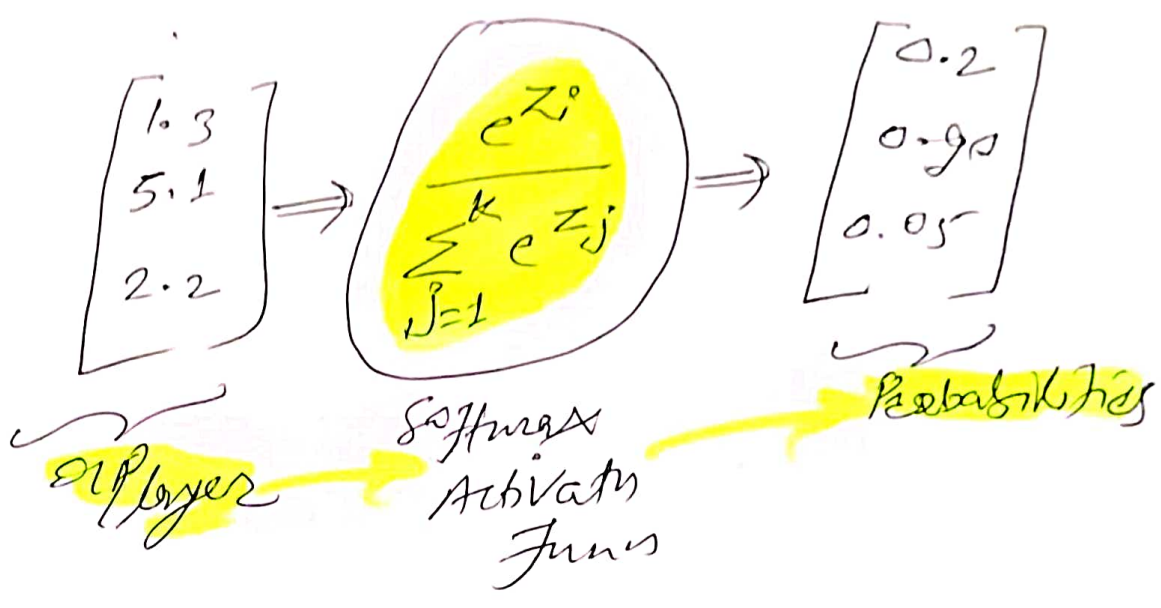
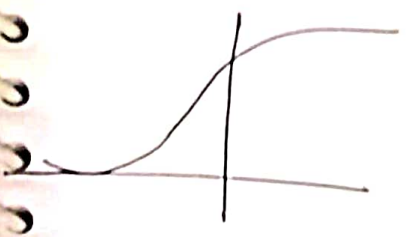


Fig $\Rightarrow$ graphs of $O(1, \log n, O(n^2), 2n,)$

② Softmax $\Rightarrow$ x in the row of the neural net into a vector of probabilities.



bias $\Rightarrow$ A constant which is added to Product of features & weights.
use to ~~shift~~ ^{help} the model ~~towards~~
to shift Activation Function towards
(+) or (-ve)
use to offset the result.

Activation Function

$\rightarrow$ It determines the output of Perceptron based on the weighted sum of IP & bias term.

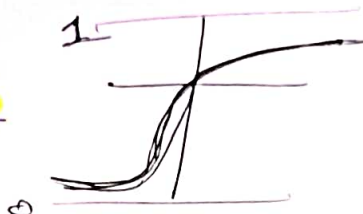
$$\text{Activation Function} (g) = \sum_j (w_j, a_j) + b_i$$

$$\text{Activation Function} = \sum (\text{weight}, \text{IP}) + \text{bias}$$

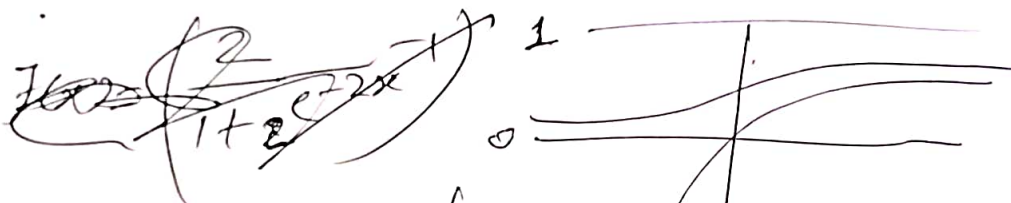
* Nonlinear Activation function are divided on the basis of their ranges or curves

① Sigmoid or logistic Activation function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



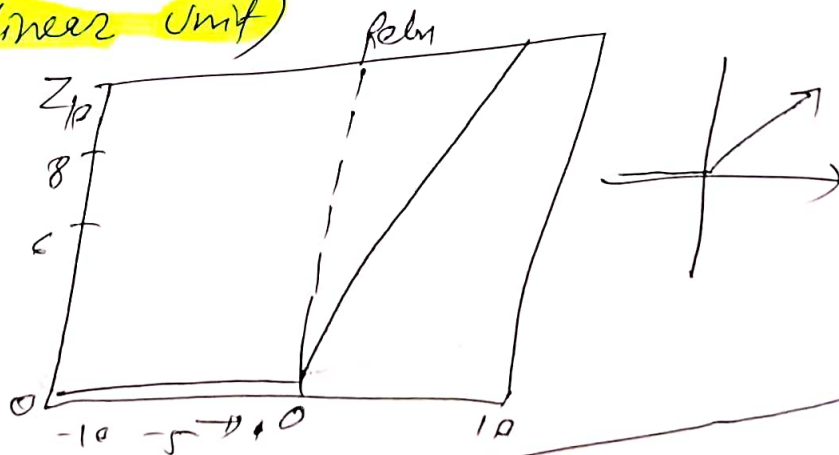
② Tanh or hyperbolic tangent Activation function (allow $(-)$ ve values)



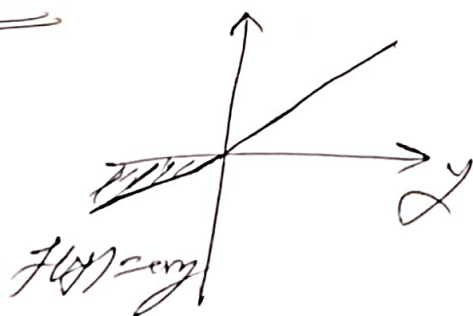
$$\text{Tanh}(x) = \left(\frac{2}{1 + e^{-2x}} \right) - 1$$

③ Relu (Rectified Linear Unit)

$$R(z) = \max(0, z)$$



④ Leaky Relu



⑤ Softplus

$$f(x) = \log(e^x + 1)$$

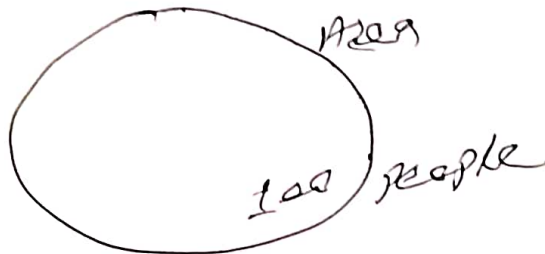


Sampling → ML

* Population

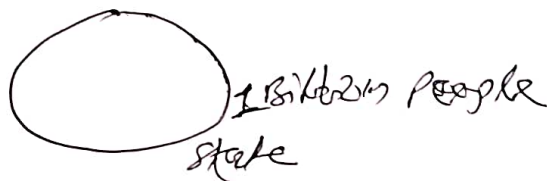
↳ All data, info, records inside db are population
Complete set

Population $\approx$ content inside db or DB
@ large info



→ find mean of veg & non-veg?

→ Population will be apply to voluminise dataset.



→ solution of population or voluminise dataset is sampling.

Sampling

taking
samples
of 10000 people



Population
of 1 Billion people's info

that samples we analyze &
conclude that based on this ~~method~~ sample
we predict about entire dataset.

∴ we conclude people are veg or non-veg.

* Sampling Techniques

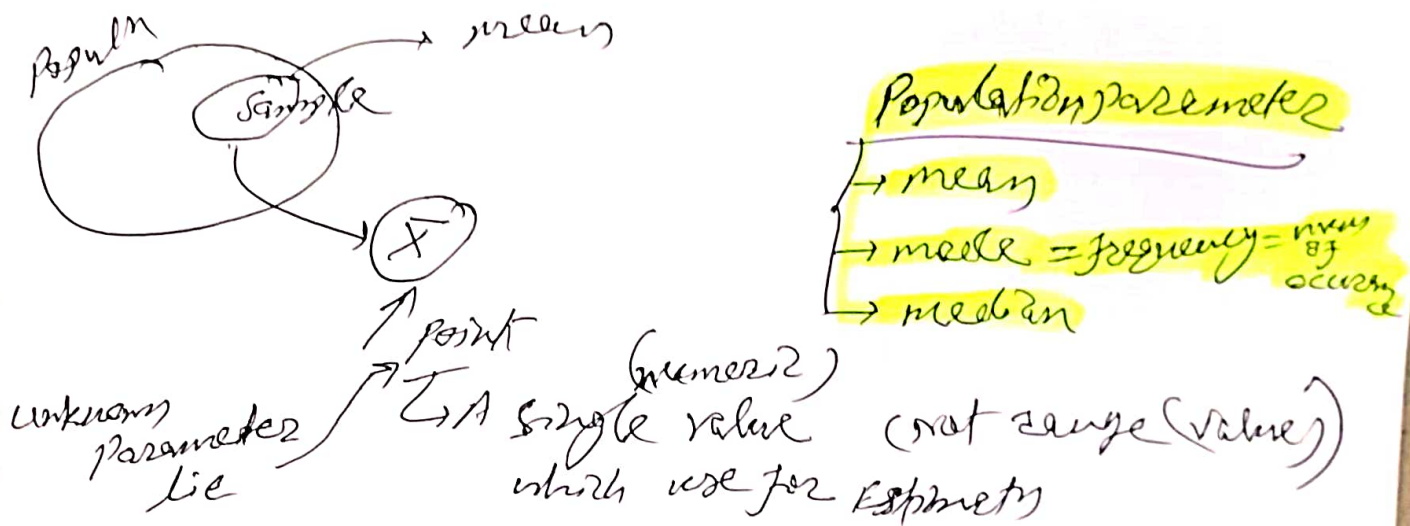
① Random

② Systematic

③ Stratified

* Point Estimate & Interval Estimate

Estimate = guess = try to reach rough values (Approx. value)



Interval → will get Range of values

Interval {100, 200}

unknown parameter lie within Range

Accuracy of interval is best compared to point estimate.

* Confidence Interval (CI)

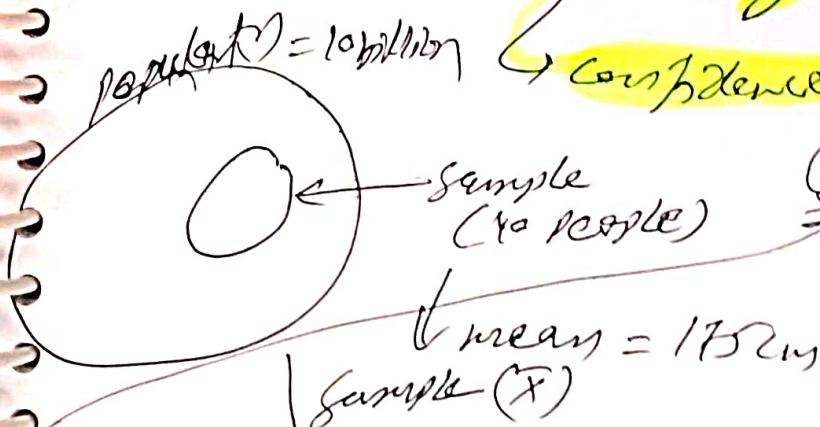
It indicate that the unknown parameter is confidently lie within the range

confidence percentage (90%, 80%, 70%, ...)

(CI = (168.8, 181.2)) 70% ...

CI = z when table

confidence = 95% = 1.960



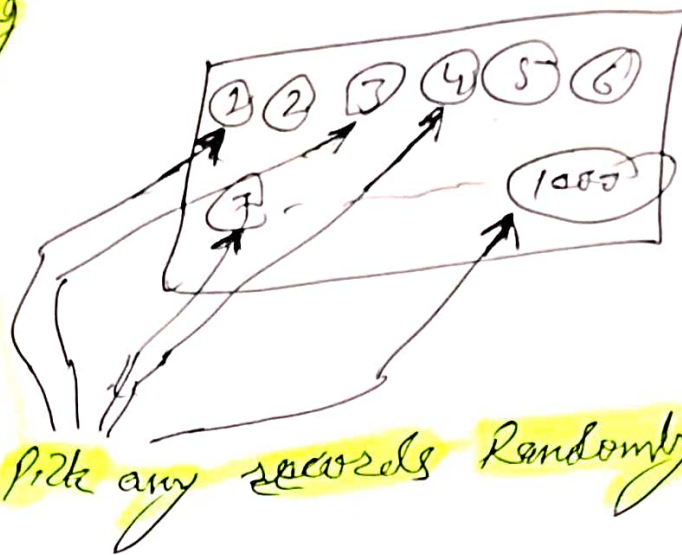
CI = 175 ± 1.960

SD = 20 cm

$$= \bar{x} \pm z \frac{SD}{\sqrt{n}}$$

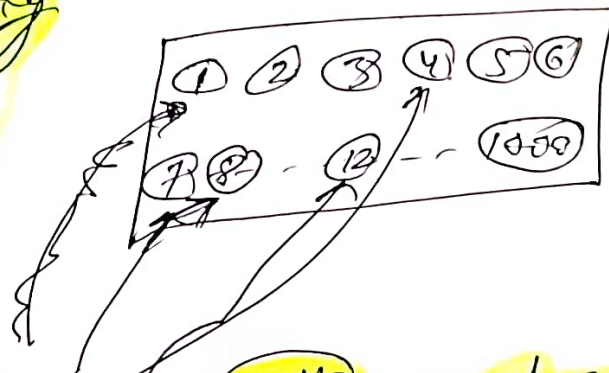
num(observe) in sample

* Random sampling



Pick any records Randomly for sampling

* Systematic sampling



Every n^{th} record get picked up for the sampling.

Every 4^{th} record selected for the sampling.

(we can change the characteristics as well)

* Stratified

Picking up data from stratum for sampling

stratum

should have some common characteristics

making subset of

a set in order to distinguish or classify.

Ex

Team 1 = even

Team 2 = odd

class of student

Ex $\rightarrow$ subset 1 = Team 1

subset 2 = Team 2

{ set = class of student }

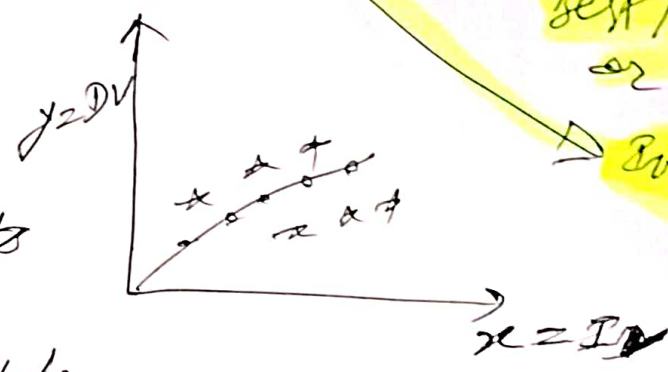
stratum

→ Coefficient of Determinant

R squared = R² value → Indicate how the best fit line is effective or good.

→ It show how effectively

the datapoints are fit to the best fit line (or datapoints)



→ Indicate how the datapoints are close to the best fit line

$$R^2 = \frac{\sum (\hat{y}_i - \bar{y})^2}{\sum (y_i - \bar{y})^2}$$

$$R^2 = \frac{\sum (\text{Predicted} - \text{mean})^2}{\sum (\text{Actual} - \text{mean})^2}$$

$\hat{y}$ = Predicted
 y = Actual
 $\bar{y}$ = y mean

R^2 = value increase, the distance betn Actual point & line are comes closer ~~to the~~
 (distance betn Actual point & line decreases)

→ If we want to place Actual point on the line then we have to increase the value of R^2

→ If $R^2 = 1$, then all values are placed on the line.

$$CI = \text{Sample mean} \pm \left(\text{Confidence \% Value from table} \right) * \frac{\text{Standard deviation}}{\sqrt{\text{num (sample)}}}$$

* MOE (Margin of Error)

↑ MOE

$$\text{MOE} = \left(\text{Confidence \% value from table} \right) * \frac{SD}{\sqrt{\text{num (sample)}}}$$

Ex: $CL = 95\% \Rightarrow 1.960$

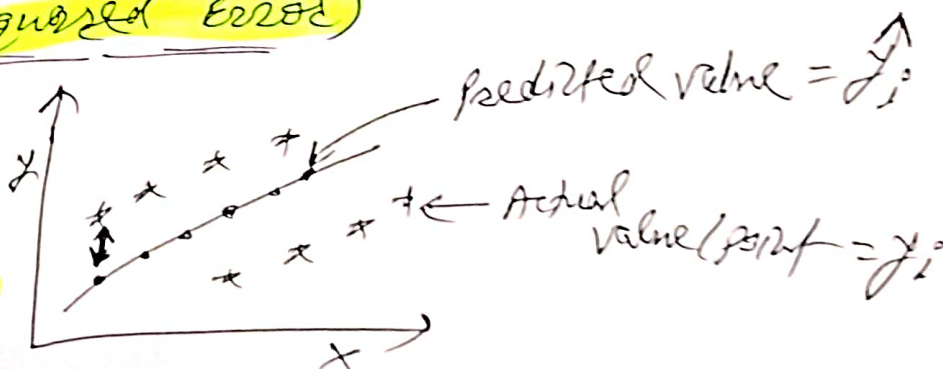
$SD = 23.44$

$n = 32$

MOE = $1.960 * \frac{23.44}{\sqrt{32}} = \underline{\underline{8.12}}$

* MSE (mean squared error)

→ lesser the distance $\hat{=}$ Better the model & Better the Accuracy



$$MSE = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$

$n = \text{total samples}$

ML with months chapters

- ① Algebra - I (Intro to Linear Algebra)
- ② Algebra - II (major opⁿ)
- ③ Calculus - I (Limits & Derivatives)
- ④ Calculus - II (Partial Derivatives & Integrals)
- ⑤ Probability & Infr system
- ⑥ Stats
- ⑦ Algo & Datastructure
- ⑧ optimizatiⁿ

python
Go PL

chapter - I (Algebra - I : Intro to Linear Algebra)

- ① Data structure for Algebra
 - ② Common Tensor opⁿ
 - ③ matrix Properties.
- } segments of units

Segment - I

- ① what is linear Algebra?
- ② A brief history of Algebra
- ③ Tensors
- ④ scalars
- ⑤ vectors & vector Transposition
- ⑥ Norms & unit vectors
- ⑦ Basis, orthogonal & orthonormal vectors
- ⑧ Array in Numpy
- ⑨ matrices

⑩ Tensors in Tensor Flow & Pytorch.

Ex $\Rightarrow$ R^2 Actual Predicted Denominator Numerator

X	Y	$\hat{Y}$	$(Y - \bar{Y})^2$	$(\hat{Y} - \bar{Y})^2$
1	4	2	0.64	1.44
2	1	3	4.84	0.04
3	3	3.5	0.04	0.09
4	5	4	3.24	0.64
5	3	4.5	0.04	1.69

mean = $\bar{Y} = 3.2$

$$\therefore R^2 = \frac{\sum (\hat{Y}_i - \bar{Y})^2}{\sum (Y_i - \bar{Y})^2} = \frac{4.3}{8.8} = 0.43$$

Note $\Rightarrow$ ^{here} mean is calculated from Actual value only

Alternatively, Problem can be solved using Algebra

$$\therefore \text{eqn 1} \Rightarrow d = 2.5t$$

$$\text{eqn 2} \Rightarrow d = 3(t - 5)$$

$$\therefore 2.5t = 3(t - 5)$$

$$2.5t = 3t - 15 - 2.5$$

$$0.5t = 15$$

$$t = \frac{15}{0.5} = \underline{30 \text{ min}}$$

$$\boxed{S = \frac{D}{T}} \Rightarrow \underline{\underline{D = ST}}$$

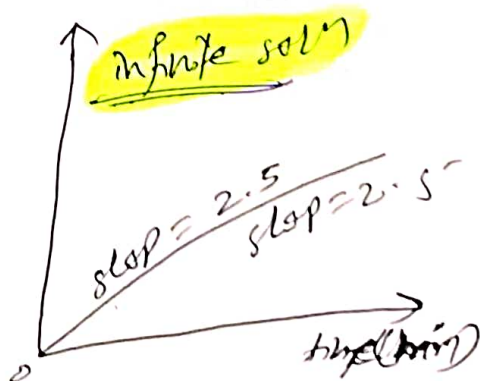
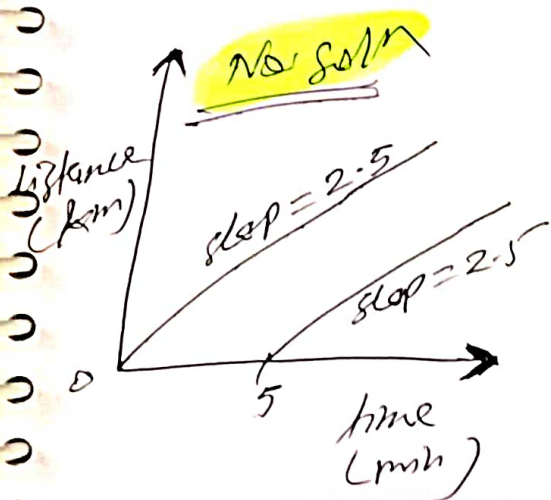
$\begin{cases} S = \text{Speed} \\ D = \text{Distance} \\ T = \text{Time} \end{cases}$

By putting 't' value in eqn ① & eqn ②

$$\text{eqn ①} \Rightarrow d = 2.5 \times 30 = 75.0 \text{ km}$$

$$\text{eqn ②} \Rightarrow d = 3(30 - 5) = 3(25) = 75 \text{ km}$$

Case - I No solution $\Rightarrow$ if Sheriff's car is same speed as bank robber's,



note $\Rightarrow$ There are the only 3 options in linear Algebra,
One soln, No soln & Infinite soln.

& It is impossible to cross lines multiple times.

① What is Linear Algebra?

→ Algebra is mathematics that includes non-numerical entities like x ,

Ex → $2x + 5 = 25$

$$2x = 25 - 5$$

$$2x = 20$$

$$x = 10$$

∴ We have determined x must be equal to 10 because $2(10) + 5 = 25$

→ Note ⇒ If any expression has an exponential term, then it is not linear algebra.

Ex → $2x^2 + 5 = 25$
 $2\sqrt{x} + 5 = 25$ } Not linear algebra

→ Defn of Linear Algebra

"Solving for unknowns within system of linear eqn"

Ex → Consider the following ex,

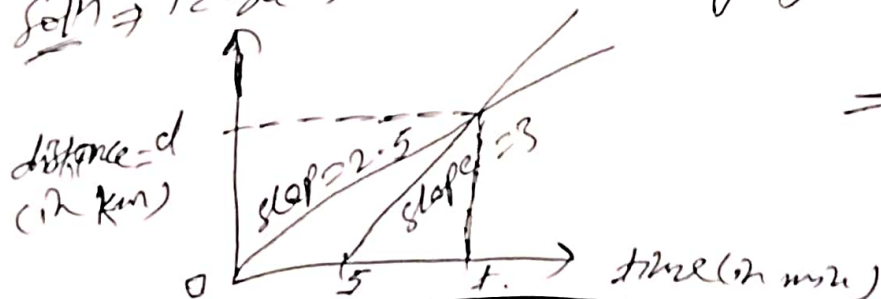
→ Sheriff has 180 km/h car

→ Bank Robber has 150 km/h car & 5-minute head start.

→ How long does it take the sheriff to catch the robber?

→ What distance will they have travelled at the road?

Soln → Problem solved using graphical plot,



Note ⇒ $150 \text{ km/h} = 2.5 \text{ km/min}$
 $180 \text{ km/h} = 3 \text{ km/min}$

II Brief History of Algebra

→ Al-Khwarizmi, mathematician (c. 780, c. 850)

→ Algebra & Algorithm 1st ~~math~~ ^{math} formulas

→ wrote the book on calculation by comp^l & balancing

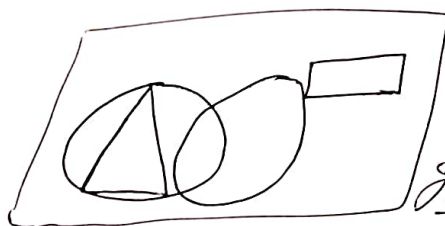
→ 1900 BC: Babylonian "algebraical"

→ 1650 BC: Egyptian

→ 6th Century BC: Indians

→ 400 - 300 BC: Greeks

→ 200 BC: Chinese



geometric Algebra
by Euclid

* Contemporary Applⁿ of Algebra

① Solving for Unknowns in ML algo, including DL.

② Reducing dimensionality (ex $\Rightarrow$ PCA = Principal Component Analysis)

③ Ranking Result (ex $\Rightarrow$ Eigen vector)

④ Recommenders (ex $\Rightarrow$ singular value decomposition, SVD)

⑤ Natural lang Processing (ex $\Rightarrow$ matrix factorization)

→ Topic modelling

→ Semantic Analysis.

* In the given system of eqn,

→ Could be many eqn

→ Could be many unknowns in each eqn

Ex → $y = a + bx_1 + cx_2 + \dots + mx_m$

Annotations:

- y : house Price
- a : "y-intercept"
- x_1 : distance to school
- x_2 : num of Bedrooms
- x_m : there could be many Features

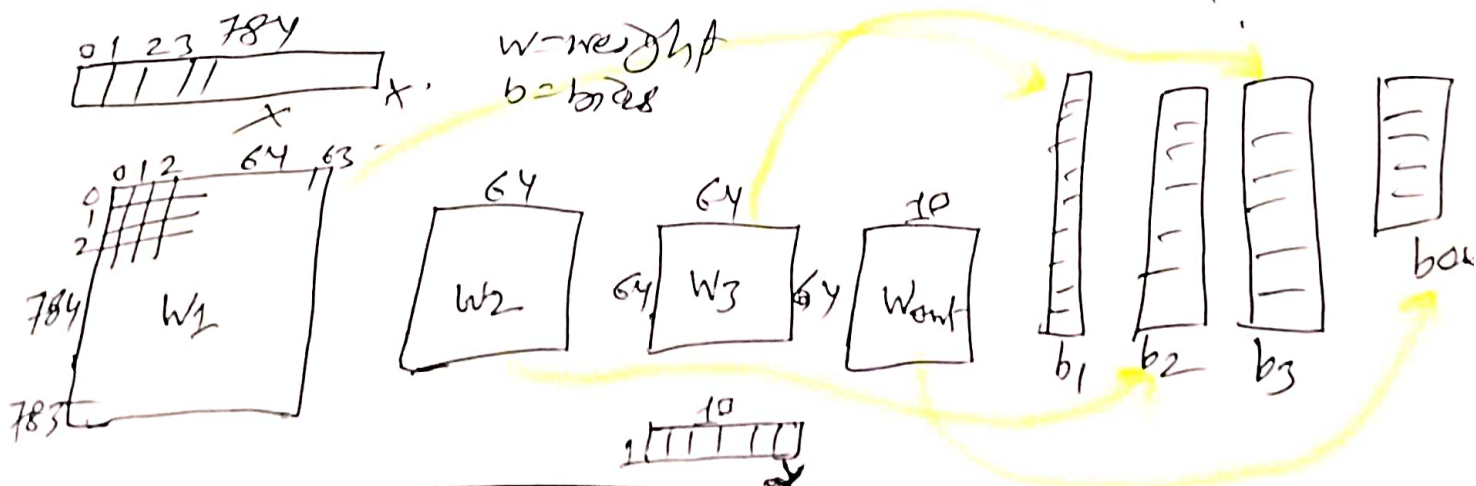
$$y = a + bx_1 + cx_2 + \dots + mx_m$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \begin{bmatrix} a + bx_{1,1} + cx_{1,2} + \dots + mx_{1,m} \\ a + bx_{2,1} + cx_{2,2} + \dots + mx_{2,m} \\ \vdots \\ a + bx_{n,1} + cx_{n,2} + \dots + mx_{n,m} \end{bmatrix}$$

For any house "i" in the dataset,

y_i = Price & $x_{i,1}$ to $x_{i,m}$ are its ~~dataset~~ Features.

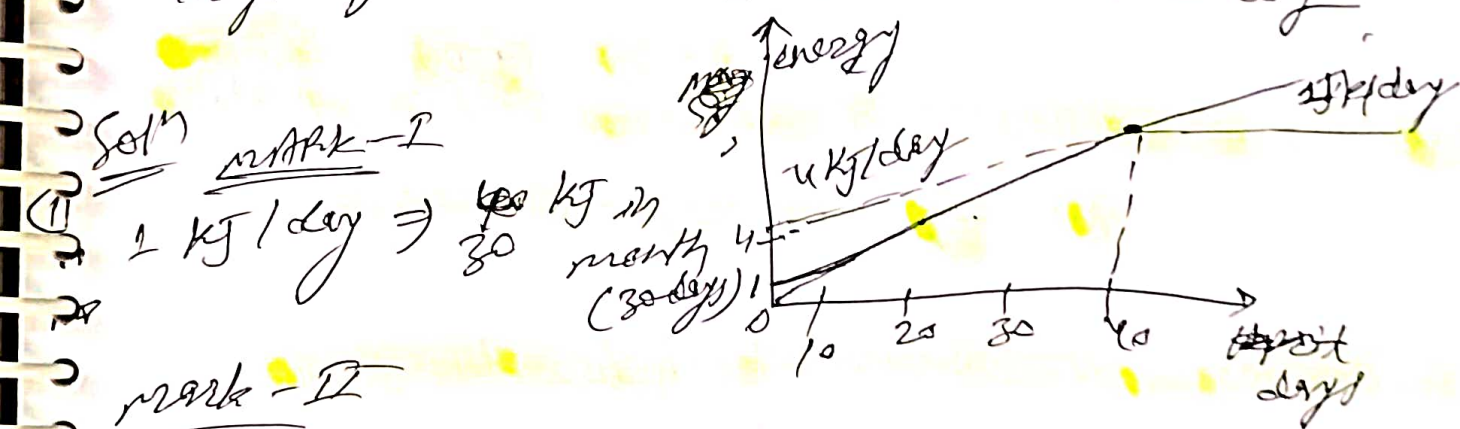
We solve for parameters a, b, c to m .



Exercise

Jill designs solar panels as a hobby,
 on April 1st, Jill's "mark-I" design begins generating
 power: 1 kJ/day .
 on May 1st, her "mark-II" design begins generating
 4 kJ of power per day.

- ① what day is it when Jill's mark-II design has generated as much total energy as the mark-I?
- ② How much total energy have both generated by that day?
- ③ what would be soln to ① & ② be if mark-II design generated 1 kJ of power per day.



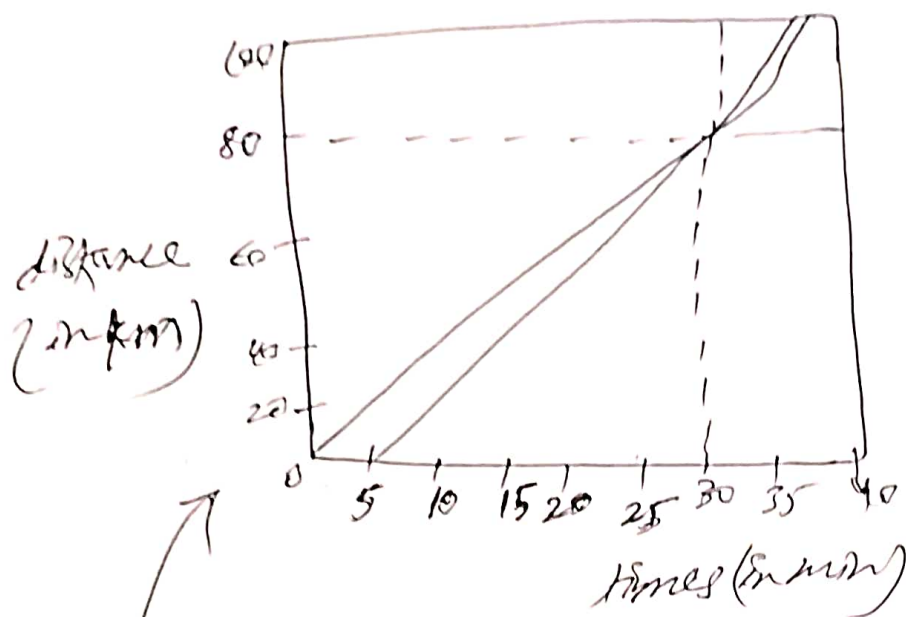
$\therefore$ Both can meet (energy of mark-I & mark-II equal)

on 40 days

- ② on 40th day $\Rightarrow$ mark-I = 40 kJ
 mark-II = 40 kJ } = 80 kJ

- ③ no solⁿ as both have equal energy generation.

A python: plotting sheriff & Bank robber as example



Code

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 40, 1000) # start, finish, n points
# distance travelled by robber  $\Rightarrow d = 2.5t$ 
d_r = 2.5 * t
# distance travelled by sheriff  $\Rightarrow d = 3(t - 5)$ 
d_s = 3 * (t - 5)

fig, ax = plt.subplots()
plt.title('A Bank robber caught')
plt.xlabel('time (in min)')
plt.ylabel('distance (in km)')
ax.set_xlim([0, 40])
ax.set_ylim([0, 100])
ax.plot(t, d_r, c='green')
ax.plot(t, d_s, c='purple', linestyle='--')
plt.axvline(x=30, color='purple', linestyle='--')
plt.axhline(y=80, color='purple', linestyle='--')
```

① Scalar Tensors

- No Dimensions
- Single numerical value
- denoted in lower case
- should be typed e.g., int, float32

Ex code → scalar (Rank 0 Tensors) in Base Python

① $x = 25$

x

o/p ⇒ 25

② $y = 3$

$py_sum = x + y$

py_sum

o/p ⇒ 28

$\left. \begin{array}{l} \text{type}(x) = \text{int} \\ \text{type}(py_sum) = \text{int} \end{array} \right\}$

③ $z = 2.5$

$p = y + z$

p

o/p ⇒ 5.5

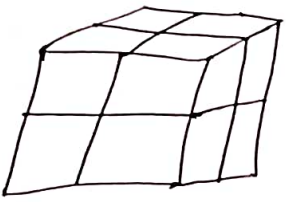
$\left. \begin{array}{l} \text{type}(p) = \text{float} \end{array} \right\}$

→ Scalar in PyTorch

- PyTorch tensors are design to be pythonic i.e., feel & behave like numpy array.
- It has advantage to easily operate on GPU.

* Tensor = Fundamental building block of Linear Algebra & any ML & DL

- Tensor flow = library in ML & DL
- Defn = It is ML generalization of Vectors & matrices to any num of dimensions.

- 0 dimension Tensor = scalar $\Rightarrow x \Rightarrow$ magnitude only
- 1 Dimension Tensor = vector $\Rightarrow [x_1, x_2, x_3] \Rightarrow$ Array
- 2 Dimension Tensor = matrix $\Rightarrow \begin{bmatrix} x_{1,1} & x_{1,2} \\ x_{2,1} & x_{2,2} \end{bmatrix} \Rightarrow$ Flat table & square
- 3 - Tensor =  $\Rightarrow$ 3D table & cube

Note $\Rightarrow$ Tensor $\hat{=}$ represent scalar, vector, matrix to any num of dimensions.

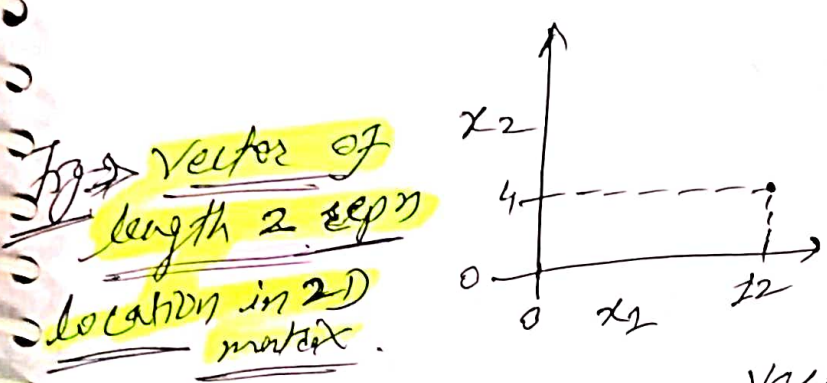
n-tensor = n-Dimensional } higher dimension

* Types of Tensors

- Scalar Tensor
- Vector Tensor
- Matrix Tensor
- 3D Tensor

* Vectors Tensor

- one-dimensional array of numbers.
- Denoted in lower case
- It arranged in order, so element can be accessed by its index.



$$[x_1, x_2] = [12, 4]$$

- Each element in vectors are scalar.
- It represent a particular point in space.
- length of 3 - represent location in 3D cube.
- length of 'n' represent location in n-dimension tensor

* Vector Transposition

$$[x_1, x_2, x_3]^T = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

row vector

column vector
shape = (3, 1)

shape = (1, 3)

Ex: Code Vectors (Rank 1 Tensors) in Numpy

import numpy as np

type(x) | x = np.array([25, 2, 5])

np.ndarray | x
shape = (25, 2, 5)

len(x)
3

x.shape()
3 = (3,)

Ex: code

import torch

x_pt = torch.tensor(25)

x_pt } ~~o/p~~ ⇒ tensor(25)

x_pt.shape

o/p ⇒ torch.size ([])

TensorFlow
PyTorch

⇒ scalar in TensorFlow

→ Tensors created with wrapper like

① tf.Variable

② tf.constant

③ tf.placeholder

④ tf.sparseTensor

Ex-code

import tensorflow as tf

x_tf = tf.Variable(25, dtype=tf.int16)

x_tf } o/p ⇒ tf.Variable(25) shape = () dtype = int16,
x_tf.shape numpy = 25

y_tf = tf.Variable(3, dtype=tf.int16)

x_tf + y_tf

tf_sum = tf.add(x_tf, y_tf)

tf_sum

tf_sum.numpy()

type(tf_sum.numpy())

* Zero vectors

→ Have no effect if added to another vector.

Ex $\Rightarrow Z = \text{np.zeros}(3)$

Z

o/p $\Rightarrow$ array ([0., 0., 0.])

Vector repⁿ the
pointer space
and magnitude &
direction.

* Vectors in pyTorch & Tensorflow

$x_{\text{-pt}} = \text{torch.tensor}([25, 2, 5])$

$x_{\text{-pt}} \}$ o/p $\Rightarrow$ tensor ([25, 2, 5])

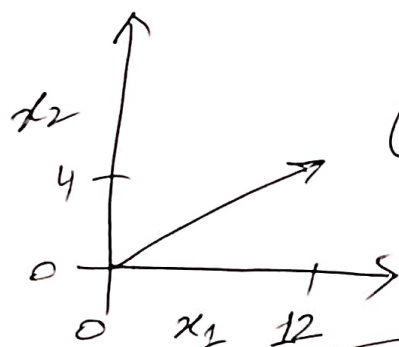
$x_{\text{-tf}} = \text{tf.Variable}([25, 2, 5])$

$x_{\text{-tf}} \}$ o/p $\Rightarrow$ <tf.Variable = 'variable:0' shape=(3,

dtype=int32, numpy=array([25, 2, 5])

dtype=int32>

* Norms = are funⁿ that quantify vector magnitude.
→ Vectors represent a magnitude & direction from origin.



$$[x_1, x_2] = [12, 4]$$

L^2 Norm

$$\Rightarrow \|x\|_2 = \sqrt{\sum_i x_i^2}$$

Absolute value

→ origin = 0

→ measure simple
(Euclidean) distance
from origin.

$$(\|x\|_2 \approx \|x\|)$$

* Vector transposition code in python

note $\Rightarrow$ Transposing a regular 1-D array has no effect

$$x_t = x.T$$

$$x_t$$

$$\text{np} \Rightarrow \text{array}([25, 2, 5])$$

$$x_t.\text{shape}$$

$$\text{np} \Rightarrow (3,)$$

But it does when we use nested "matrix style" brackets

$$y = \text{np.array}([[25, 2, 5]])$$

$$y$$

$$\text{np} \Rightarrow \text{array}([[25, 2, 5]])$$

$$y.\text{shape}$$

$$\text{np} (1, 3)$$

note $\Rightarrow$ we can transpose a matrix with dimension of length 2 which is mathematically equivalent,

$$y_t = y.T$$

$$y_t$$

$$\text{np} \Rightarrow \text{array}([[25], [2], [5]])$$

$$y_t.\text{shape}$$

$$\text{np} \Rightarrow (3, 1)$$

Squared L^2 Norm

$$\|x\|_2^2 = \sum_i x_i^2$$

It is computationally cheaper to use than L^2 norms.

because, $\boxed{\text{Squared } L^2 \text{ Norm} \equiv \text{tr } X^T X}$

Why is it, it grows slowly near origin, $\therefore$ Can't be used to distinguish betn zero & non-zero.

Ex $\Rightarrow x = 0/p ([25, 2, 5])$

$(25^2 + 2^2 + 5^2)$

$\hookrightarrow \text{O/p} = 654$

$\text{np.dot}(x, x)$

$\hookrightarrow \text{O/p} = 654$

L^2 use to Regularize objective funn

L^∞ Norm $\equiv \max \text{ Norm} \Rightarrow$ It return the absolute value of the largest-magnitude element.

$$\|x\|_\infty = \max_i |x_i|$$

Ex $\Rightarrow \text{np.max}([\text{np.abs}(25), \text{np.abs}(2), \text{np.abs}(5)])$

$\hookrightarrow \text{O/p} = 25$

Generalized L^p Norm $\Rightarrow$ can derive L^1, L^2 & L^∞ by 'p' Substituting

$$\|x\|_p = \left(\sum_i |x_i|^p \right)^{1/p}$$

$\rightarrow p = \text{must be real number}$
 $\& p \geq 1$

Ex $\rightarrow x$ 3D = array $([25, 2, 5])$

$$(25 \times 2 + 2 \times 2 + 5 \times 2) \times (1/2)$$

$\rightarrow$ 3D $\Rightarrow 25.573423$

np.linalg.norm(x)

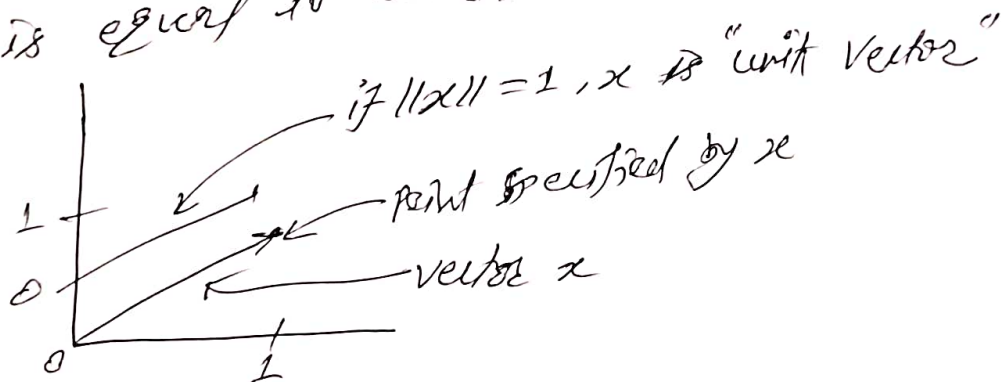
$\rightarrow$ 3D $\Rightarrow 25.573423$

$\therefore$ if units in this 3-Dimensional vector space are meters, then the vector x has a length of 25.6m

* Unit Vectors

$\rightarrow$ special case of vectors where its length is equal to one.

$$\|x\| = 1$$



L1 Norms = It varies linearly at all locations whether near or far from origin

$$\|x\|_1 = \sum_i |x_i|$$

$\rightarrow$ It is used whenever difference betn zero & non-zero is key.

Ex $\rightarrow x$ 3D $\Rightarrow$ array $([25, 2, 5])$

$$\text{np.abs}(25) + \text{np.abs}(2) + \text{np.abs}(5)$$

$\rightarrow$ 3D $\Rightarrow 32$

Ex $\Rightarrow$ orthogonal vectors

$i = \text{np.array}([1, 0])$

$i \text{ } \{ \text{o/p} \Rightarrow \text{array}([1, 0])$

$j = \text{np.array}([0, 1])$

$j \text{ } \{ \text{o/p} \Rightarrow \text{array}([0, 1])$

$\text{np.dot}(i, j)$

$\{ \text{o/p} \Rightarrow \underline{\underline{0}}$

0D = scalar Tensor

1D = Vector Tensor

2D = matrices
Tensor

* Matrices

$\rightarrow$ 2D array of num

denoted in uppercase X

Height (row) given priority
ahead of width in notation
(column)

$x_{1,1}$	$x_{1,2}$	25	2
$x_{2,1}$	$x_{2,2}$	5	26
$x_{3,1}$	$x_{3,2}$	3	7

(3x2)

string $\Rightarrow x[0:2, 0:2]$
array $\Rightarrow \text{array}([25, 2], [5, 26])$

Individual scalar elements denoted in uppercase (state only)

Ex $\Rightarrow x = \text{np.array}([25, 2], [5, 26], [3, 7])$

$x \text{ } \{ \text{o/p} \Rightarrow \text{array}([25, 2], [5, 26], [3, 7])$

$x.\text{shape} \{ \text{o/p} \Rightarrow (3, 2)$

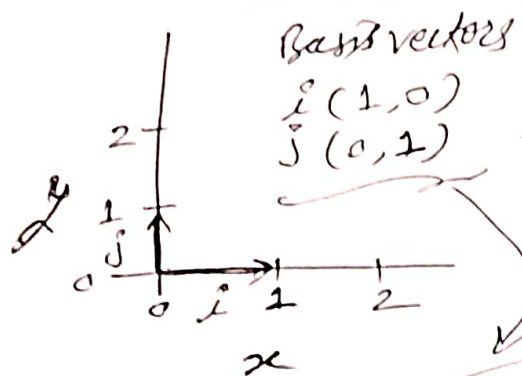
$x.\text{size} \{ \text{o/p} \Rightarrow 3 \times 2 = \underline{\underline{6}}$

(left column)
 $x[:, 0] = \text{array}([25, 5, 3])$

$x[1, :] = (5, 26)$
(middle row)

* Basis Vector → can be scaled to represent any vector in given vector space
 ↳ it uses unit vectors

(16)

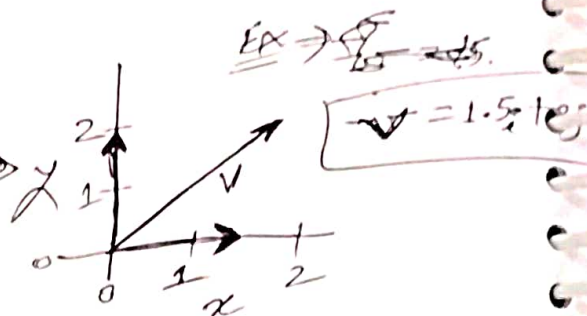


Basis vectors

$$\hat{i}(1,0)$$

$$\hat{j}(0,1)$$

Can be scaled to any Vector



v = vector

Basis vector
 unit vectors

$\hat{i}(x,y)$
 $\hat{j}(x,y)$

* Orthogonal Vectors

↳ x & y are orthogonal vectors, if $\{x^T y = 0\}$

dot product

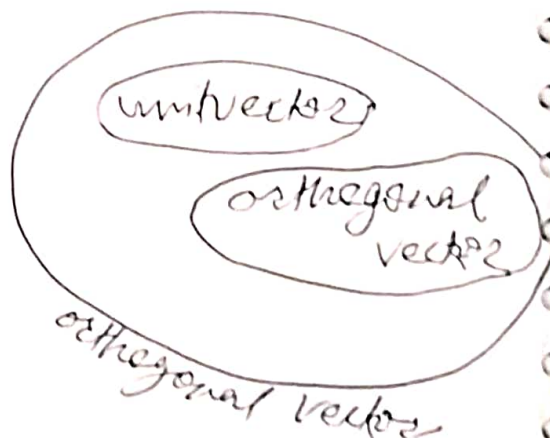
↳ Both x & y are at Angle 90° to each other (Assuming non-zero norms)

↳ n -dimensional space has max ' n ' mutually orthogonal vectors

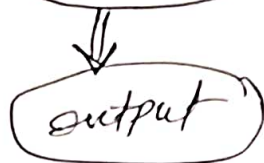
* Orthogonal

↳ orthogonal vectors are both orthogonal & unit vector

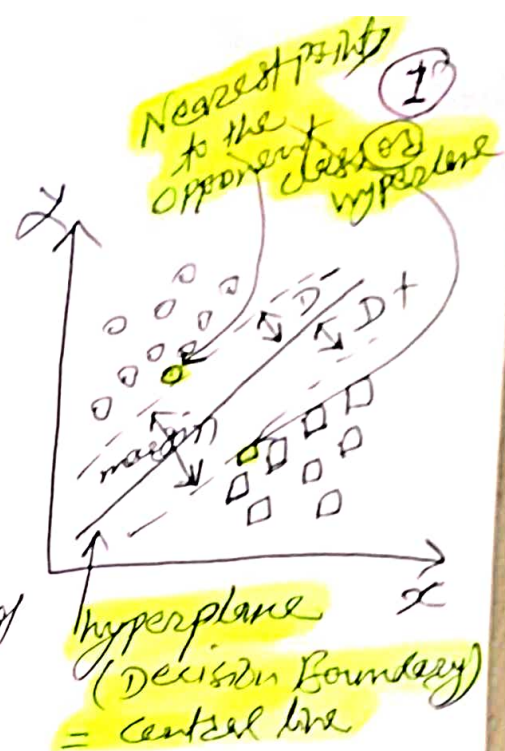
Basis vector $\Rightarrow$ orthogonal vector
 are example of



* Support Vector m/c



□ ← quadrilaterals
○ ← circle



Problem defⁿ ⇒ data points belongs to which class of category.

hyperplane = decision boundary ⇒ It divides or indicate the new data belongs to which class of category.

$D^+, D^- \equiv$ distance betn hyperplane & closest datapoint

↑ distance (−)ve distance

margin = sum of distances D^+ & D^- (or) distance betn closest datapoints of both the classes.

or $\boxed{\text{margin} = D^+ + D^-}$

Support Vector = closest datapoints of both classes based on which the decision boundary is drawn.

A matrices in PyTorch

`x_pt = torch.tensor([25, 2], [5, 26], [3, 7])`

`x_pt`
 $\hookrightarrow$ app $\Rightarrow$ tensor([25, 2], [5, 26], [3, 7])

`x_pt.shape`

$\hookrightarrow$ app $\Rightarrow$ torch.size([3, 2])

`tf.rank(x_tf)`

$\hookrightarrow$ app = `<tf.Tensor : shape=(), dtype=int32, numpy=2>`

Numericals of Linear Regression

Σ = summation

Question $\Rightarrow$

X	Y	xy	X ²
1	3	3	1
2	4	8	4
3	5	15	9
4	7	28	16
10	19	54	30

$$a = \frac{(\Sigma Y)(\Sigma X^2) - (\Sigma X)(\Sigma XY)}{n(\Sigma X^2) - (\Sigma X)^2}$$

$\therefore n = \text{num (rows)}$
or num (observ)

$$b = \frac{n(\Sigma XY) - \Sigma X \Sigma Y}{n(\Sigma X^2) - (\Sigma X)^2}$$

$Y = bX + a$

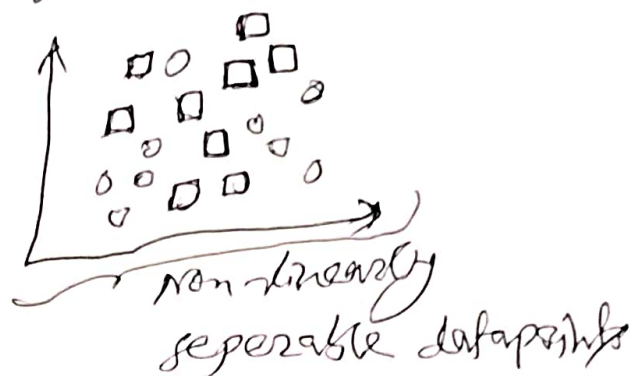
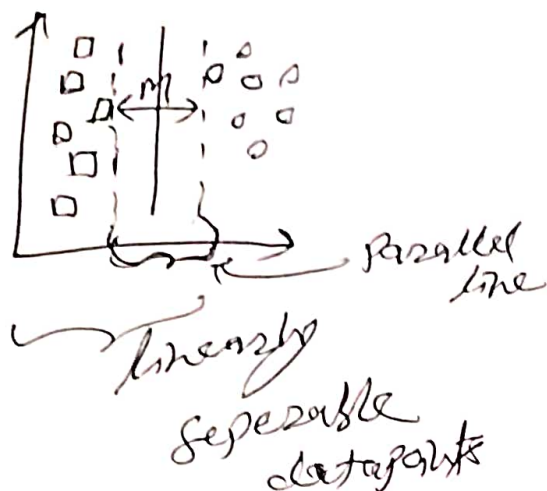
$a = \frac{19 \times 30 - 10 \times 54}{4 \times 30 - 100} = \frac{570 - 540}{120 - 100} = \frac{30}{20} = 1.5$

$b = \frac{4 \times 54 - 10 \times 19}{120 - 100} = \frac{216 - 190}{20} = \frac{26}{20} = \frac{13}{10} = 1.3$

$\therefore Y = 1.3X + 1.5$

* Linearly separable data points

→ which separate by drawing any line



⇒ linearly separable data points given to linearly support Vector machine.

⇒ margin should be of maximum width

→ select that SVM.

(maxim) maximal margin Hyperplane

→ It helps to predict well & gives accurate result.

→ reduce the error rate.

→ reduce the possibility of mis-classification.

Error Rate ↓ $\approx$ Accuracy ↑
less Rate higher

Why overfitting happens?

Train = preparing
Test = Validation

- Training data size is too small.
- Training data set does not contain enough data samples to accurately represent all possible i/p data values.

Why underfitting happens?

- when a model is too simple, which can be result of model needing more training time, more i/p features or less regularization.

How to know overfitting?

model performs ^{well} on training set but poorly on the validation set.

Tips to Reduce over & underfitting

- 1) select Relevant features.
- 2) use Regularization to prevent overfitting
- 3) Employ cross-validation for model evaluation.

4) Explore ensemble learning model

5) fine-tune model hyperparameters

^{hyper}
(parameters = learning rate, regular strength, tree depth)

MSE & Accuracy are used to evaluate

→ grid search

→ Randomized search

→ Balance Bias & Variance

→ splitting data into multiple subsets & perform iterative training & evaluating the model on different combn of training & validation set.

→ check for different values of x & find $P = \text{Predicted value}$

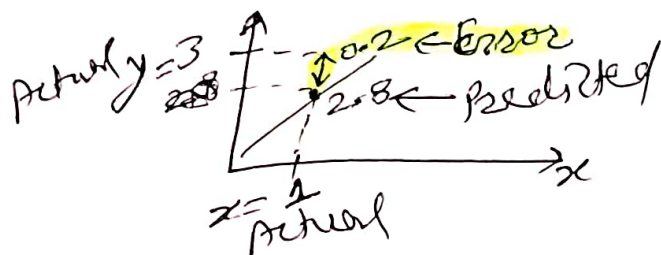
$$Y = 1.3X + 1.5$$

$E = \text{Error rate}$

Let $X = 1$,

$$\therefore Y = 1.3 \times 1 + 1.5 = 2.8$$

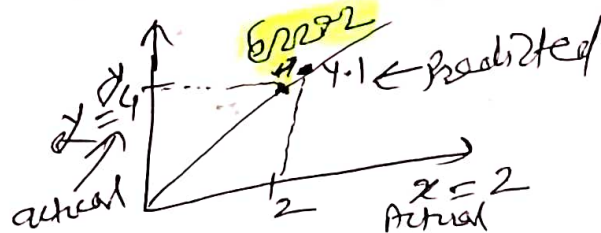
$$\begin{cases} P = 2.8 \\ E = \text{Actual value} - \text{Predicted value} \\ = 3 - 2.8 \\ = \underline{\underline{0.2}} \end{cases}$$



Let $X = 2$,

$$\therefore Y = 1.3 \times 2 + 1.5 = 2.6 + 1.5 = 4.1$$

$$\begin{cases} P = 4.1 \\ E = 4.1 - 4 \\ = \underline{\underline{0.1}} \end{cases}$$



Let $X = 3$,

$$\therefore Y = 1.3 \times 3 + 1.5 = 3.9 + 1.5 = 5.4$$

$$\begin{cases} P = 5.4 \\ E = 5.4 - 5 = \underline{\underline{0.4}} \end{cases}$$

Let $X = 4$,

$$\therefore Y = 1.3 \times 4 + 1.5 = 5.2 + 1.5 = 6.7$$

$$\begin{cases} P = 6.7 \\ E = 6.7 - 7.0 \\ = \underline{\underline{0.3}} \end{cases}$$

Ref. Table on previous page

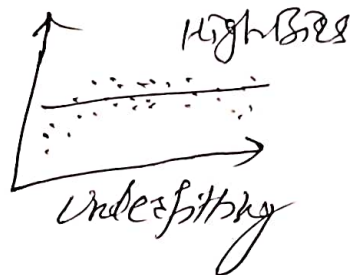
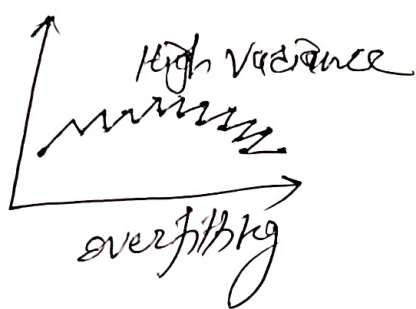
x	y	xy	x^2	P	E
1	3	3	1	2.8	0.2
2	4	8	4	4.1	0.1
3	5	15	9	5.4	0.4
4	7	28	16	6.7	0.3
10	19	54	30		

* Regularization

→ It is a technique used to reduce errors by fitting the function appropriately on the given training set & avoiding overfitting

Techniques

- ① Lasso Regr $\cong$ L1 Regularization
- ② Ridge Regr $\cong$ L2 Regularization
- ③ Elastic Net Regr $\cong$ L1 & L2 Regularization



② Lasso Regr $\Rightarrow$ L1 Regr $\Rightarrow$ (Least Absolute shrinkage & selection)

→ It adds the Absolute value of magnitude

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$

$m = \text{num (features)}$

$n = \text{num (examples)}$

$y_i = \text{Actual target value}$

$\hat{y}_i = \text{Predicted target value}$

Bias = distance from target

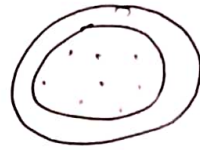
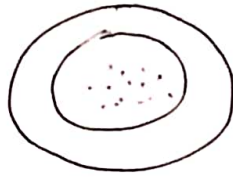
variance = change in scatterness

Best ml model

Low variance

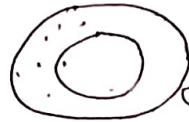
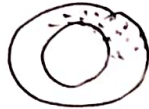
High variance

Low Bias



overfitting

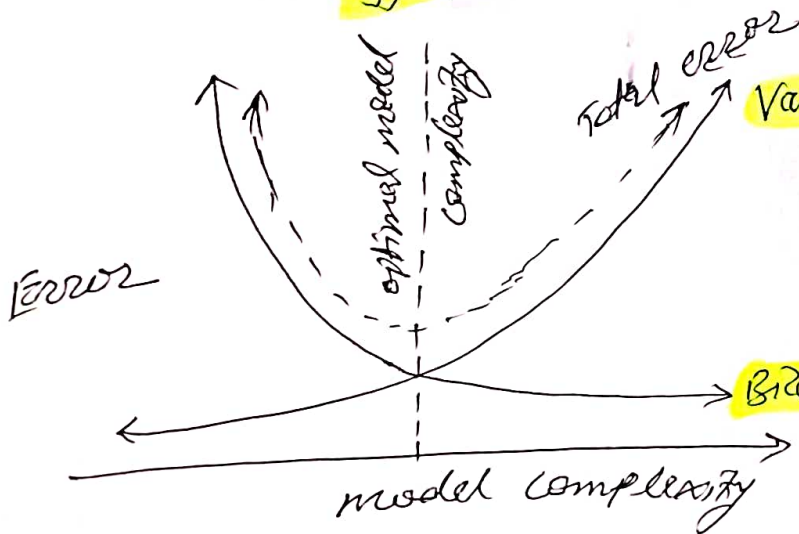
High Bias



unreliable & inconsistent predictions

underfitting

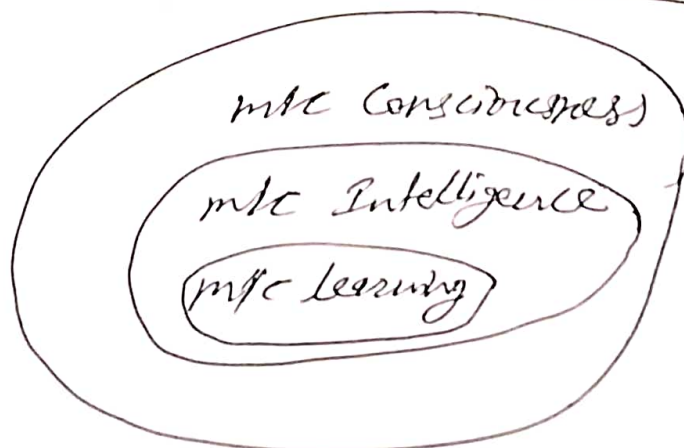
Bias-variance tradeoff = Balance betn bias & variance, which affect predictive model & performance.



variance = change in model when using different portions of training data

Bias = error that occurs in ml due to incorrect assumption

stages of AI



* Bias & Variance

→ Refers to the gap betn Predicted & Actual value.

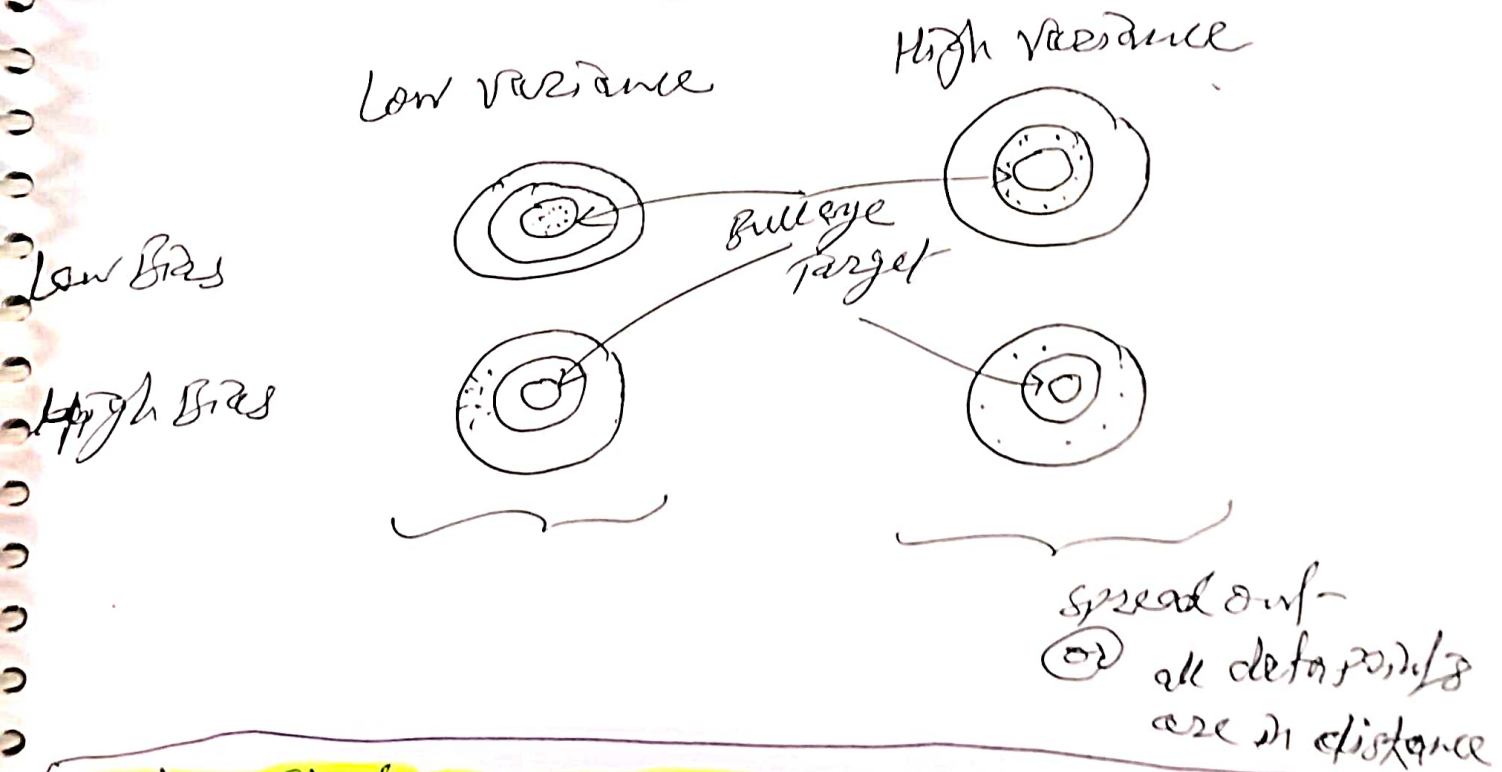
→ high Bias = Predicted value is far away from actual value

→ low Bias = Predicted value is away but not so away from actual value

Variance = how much scattered the predicted values are in crowded or collective share space

→ low variance = All predicted values are spread &

→ high variance = All predicted values are spread & away from each other.



Note ⇒ Ideal case = Low Bias & Low variance

our objective should be to reach the ideal case.

② Ridge Regn

it adds squared magnitude of coefficient

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

③ Elastic Net Regn = combⁿ of L1 & L2

it adds weights & squared magnitude of weights.

need an extra hyperparameter.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1-\alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2 \right)$$